LLM Systems Reader: Math Refresher

Companion to *Large Language Models as Engineered Systems*

May 10, 2026

Contents

How to Use This Companion

Use this as a lookup tool. Read only the chapter you need for the reader chapter you are working through. If a chapter points to an earlier refresher chapter, follow that pointer instead of re-reading repeated material. The chapter numbers here match those of the main reader exactly.

Preface: Probability Building Blocks

How to read this preface

The chapters ahead assume undergraduate probability fluency that may have faded. This preface does not replace a textbook; it reinstalls a *Strang-style chain*: each subsection states what problem the next definition solves, names where the idea entered mathematics or engineering, and ends with a *mental model* you will reuse when reading LLM systems notation.

Probability here means Kolmogorov-style measure on a sample space. Random variables compress events into summaries (numbers or vectors). Expectation averages those summaries under a distribution. Information quantities reinterpret probabilities as coding costs. Large-sample intuition connects finite training batches to population quantities. Later refresher chapters pointer-reference these steps instead of repeating them.

Step 1 — Sample space, events, and axioms

Problem. We must assign consistent numbers to uncertain outcomes so algebraic manipulation respects logic.

Construction (compact). A sample space Ω collects mutually exclusive elementary outcomes. An event is a subset $A \subseteq \Omega$ (in the σ-algebra $\mathcal{F}$ when rigor demands). A probability measure P obeys $P(\Omega) = 1$, $P(A) \geq 0$, and countable additivity on disjoint unions.

Insight. Uncertainty becomes bookkeeping: combine events with set operations; probabilities inherit monotone laws that mirror everyday implication.

Lineage. Kolmogorov's axioms (1933) unified earlier paradox-prone intuitions into measure theory—still the lingua franca of ML papers.

Mental model. Before manipulating symbols, write down what Ω *is*. Ambiguous Ω produces ambiguous conditioning later—especially when "token" vs "character" vs "byte" outcomes differ.

Step 2 — Conditional probability

Problem. Observations arrive partially: we learn B happened and must revise beliefs about A.

Definition. For $P(B) > 0$,

$$P(A \mid B) = \frac{P(A \cap B)}{P(B)}.$$

Equivalently $P(A \cap B) = P(A \mid B)P(B)$.

Insight. Conditioning *restricts and renormalises* the world to the slice where B is true.

Lineage. Formalised within Kolmogorov's programme; conceptual ancestors include Bernoulli/Huygens gaming mathematics and later Bayesian narratives.

Mental model. Language-model "next token given context" *is* conditional probability: B is "we saw prefix $x_{<t}$," A is "next symbol equals x_t."

Step 3 — Chain rule for probabilities

Problem. High-dimensional joints look unmanagable unless we factor them along an ordering.

Identity. Always $P(A, B) = P(A \mid B)P(B)$. Iterating along an ordered sequence gives

$$P(x_{1:T}) = \prod_{t=1}^{T} P(x_t \mid x_{<t}),$$

provided each conditional is defined (positive-probability prefixes).

Insight. Sequential prediction needs no extra axiom beyond Step 2—factorisation is algebraic honesty.

Lineage. Elementary consequence of definitions; Markov (1913) showcased sequential dependence in text before computers existed.

Mental model. Each factor answers "what fraction of surviving probability mass lands on this symbol *after* seeing the left context?" Misordered dependencies confuse causality with notation.

Step 4 — Bayes' theorem (compact)

Problem. Sometimes we observe effects and must infer latent causes or revise hypotheses.

Form. With $P(E) > 0$, $P(H \mid E) = \dfrac{P(E \mid H)\,P(H)}{P(E)}$.

Insight. Invert conditioning direction by paying the evidence normaliser $P(E)$.

Lineage. Bayes; Laplace championed inferential uses.

Mental model. Pure LM training rarely expands Bayes in full, but calibration papers and Bayesian interpretations of weight decay/noise often invoke posterior language—recognise when authors silently swap θ and data roles.

Step 5 — Random variables and expectation

Problem. Events are verbose; we want numerical summaries whose averages behave linearly.

Discrete expectation.

$$\mathbb{E}[f(X)] = \sum_{x \in \mathcal{X}} p(x)\, f(x).$$

Linearity: $\mathbb{E}[af(X) + bg(X)] = a\,\mathbb{E}[f(X)] + b\,\mathbb{E}[g(X)]$ (no independence needed).

Insight. Expectation is the "center of mass" of outcomes weighted by probability—the quantity laws of large numbers later approximate.

Lineage. Nineteenth-century mechanics analogies → measure-theoretic integral (Lebesgue).

Mental model. Empirical cross-entropy is a finite-sample proxy for $\mathbb{E}_{x\sim\text{data}}[-\log q_\theta(x)]$; stochastic gradients perturb that proxy.

Step 6 — Variance and sums (sketch)

Problem. Expectation hides spread; optimisation noise depends on second moments.

Definitions. $\operatorname{Var}(X) = \mathbb{E}[(X - \mathbb{E}[X])^2]$; for independent X, Y, $\operatorname{Var}(X+Y) = \operatorname{Var}(X) + \operatorname{Var}(Y)$.

Insight. Independent minibatch gradients sum variances—batch scaling trades compute against noise amplitude.

Lineage. Classical; Chebyshev's inequality links variance to tail bounds.

Mental model. When inputs correlate inside a batch, $\operatorname{Var}(\sum)$ gains covariance terms—clip-heavy regimes acknowledge heavy tails violating naive Gaussian pictures.

Step 7 — Law of large numbers (operational)

Problem. Finite datasets nonetheless approximate unseen distributions—when is that morally justified?

Qualitative statement. Under IID or weakened dependence, sample averages $\frac{1}{n}\sum_{i=1}^{n} f(X_i)$ converge to $\mathbb{E}[f(X)]$ as $n \to \infty$ (modes differ: weak vs strong LLN).

Insight. Training losses reference an underlying population quantity even though only finite corpora arrive.

Lineage. Early probabilists (Bernoulli); modern proofs measure-theoretic.

Mental model. Engineering monitors finite-n deviation (bias, variance, drift); LLM corpora violate IID—LLN intuition guides, assumptions rarely hold verbatim.

Step 8 — Maximum likelihood

Problem. Choose parameters θ so observed data look plausible under P_θ.

Objective. $\hat{\theta} \in \arg\max_\theta \prod_{i=1}^{n} P_\theta(\text{datum}_i)$; equivalently maximise $\sum_i \log P_\theta(\text{datum}_i)$.

Insight. ML estimation aligns probabilities with frequencies inside the model class—not proof of truth outside that class.

Lineage. Fisher sharpened theory in the 1920s–1930s; modern deep learning uses regularised variants.

Mental model. Products become sums via log—derivatives and curvature numerics prefer sums; "negative log-likelihood" is the minimisation convention.

Step 9 — Shannon entropy

Problem. Quantify uncertainty with a single scalar aligned with coding limits.

Definition (discrete). $H(p) = -\sum_x p(x) \log p(x)$ (nats if ln, bits if $\log_2$).

Insight. Optimal lossless codes for draws from p require $\approx H(p)$ symbols per draw on average—entropy is not mysticism, it is asymptotic compression cost.

Lineage. Shannon's *A Mathematical Theory of Communication* (1948).

Mental model. Never compare entropies computed in different log bases without conversion; tokenisation changes $|\mathcal{X}|$ and hence typical H magnitudes.

Step 10 — Cross-entropy and Kullback–Leibler divergence

Problem. Models q disagree with reference p; quantify mismatch in bits/nats.

Definitions.

$$H(p, q) = -\sum_x p(x) \log q(x),$$

$$D_{\mathrm{KL}}(p\|q) = \sum_x p(x) \log \frac{p(x)}{q(x)}.$$

Decomposition. $H(p, q) = H(p) + D_{\mathrm{KL}}(p\|q)$.

Insight. Cross-entropy splits unavoidable uncertainty ($H(p)$) plus excess coding penalty from using q when truth is p (the KL term).

Lineage. Shannon coding theorems motivate $H(p, q)$; Kullback and Leibler (1951) formalised D_{KL}.

Mental model. Minimising sample cross-entropy pushes q toward empirical token frequencies *inside the softmax parameterisation*; KL penalties in alignment explicitly drag policies toward references—same symbolic animal, different rhetorical frame.

Where Chapter 1 picks up

Chapter 1 of this refresher specialises Steps 2–3 to autoregressive tokens, Steps 8–10 to softmax logits and calibration diagnostics, and binds Step 7 qualitatively to dataset-scale claims. Re-read this preface whenever conditioning or information identities feel like unmotivated algebra.

Chapter 1

Chapter 1 Refresher: Statistical Learning Foundations for Large Language Models — Probability, Likelihood, and Calibration

SETTING THE SCENE

Chapter 1 of the reader applies the probability spine summarised in the *Preface: Probability Building Blocks*: Steps 2–4 turn sequential text into products of conditionals; Steps 8–10 justify cross-entropy training as likelihood plus coding intuition; Step 7 motivates interpreting finite corpora as imperfect proxies for expectations.

Reasoning cadence (Strang-style). Each numbered derivation below answers an explicit question that bothered practitioners historically:

1. **What object is "predict next token"?** Conditional probability (restrict and renormalise to context).
2. **How do those pieces assemble into a story probability?** Iterated conditioning—Markov's linguistic intuition modernised.
3. **How do we fit parameters?** Fisher-style maximum likelihood ⇒ sums of log probabilities.
4. **How do we interpret the scalar loss?** Shannon/Kullback–Leibler information decomposition separates irreducible noise from model mismatch.
5. **Why softmax logits?** Analytic convenience plus numerical stabilisation via shift invariance (engineering cousin of log-sum-exp proofs).
6. **What does calibration add that loss omits?** Reliability of stated probabilities versus empirical hit rates (psychometrics meets scoring rules).

The critical insight for Chapter 1 is that language modeling does not invent probability theory; it *sequences* classical moves under computational constraints. Logarithms convert products into sums because curvature and gradient variance behave better that way—not because theoreticians prefer prettier notation.

This refresher chapter specialises the preface steps into LM notation; reread the preface whenever conditioning identities feel unmotivated.

Notation Introduced in This Chapter

- x_t: token at position t in a sequence; $x_{<t}$ denotes all preceding tokens (the context).
- V: vocabulary; $|V|$ is vocabulary size.
- $p_\theta(\cdot \mid x_{<t})$: model's conditional distribution over next tokens given context.
- $z_t \in \mathbb{R}^{|V|}$: vector of logits before softmax at position t.

What You Need To Recall Precisely

- **Conditional probability as prediction primitive.** If A and B are events with $P(B) > 0$, then
$$P(A \mid B) = \frac{P(A \cap B)}{P(B)}.$$
For language modeling, A is "next token equals x_t" and B is "context is $x_{<t}$". *Insight.* Conditioning formalises "given everything written so far." *Lineage.* Kolmogorov-era formalism; psychological predecessors debated inverse probability for centuries. *Mental model.* Always ask what slice of Ω you conditioned on—ambiguous context strings yield ambiguous measures.

- **Chain rule as sequence constructor.** Start from the definition of conditional probability in joint form,
$$P(A, B) = P(A \mid B)\, P(B).$$
Identify (A, B) with "token x_T takes its realised value *and* prefix $x_{<T}$ takes its realised values". Then $P(x_{1:T}) = P(x_T \mid x_{<T})\, P(x_{<T})$. Peel the same identity off $P(x_{<T})$ to introduce $P(x_{T-1} \mid x_{<T-1})$, and continue until only $P(x_1)$ remains. Induction yields
$$p(x_{1:T}) = \prod_{t=1}^{T} p(x_t \mid x_{<t}).$$
Reading direction: first factor is "probability of x_1 before anything is observed" (often simplified under a start token convention); each later factor is "next-token prediction given everything to the left". This is *not* a modeling trick—it is bookkeeping forced by the definition of conditional probability once you adopt an ordering on positions. *Insight.* Long-range dependence enters only through what you put inside each conditional—the factorisation itself assumes nothing about Markov structure. *Lineage.* Used implicitly by Markov (1913); standard autoregressive NLP since Bengio et al. (2003) and contemporaries. *Mental model.* If a factor equals zero, later narrative probability collapses—numerical underflow in logs mirrors this.

- **Likelihood to optimization objective.** Maximum likelihood chooses parameters that maximize probability of observed data. Because products are awkward numerically, we apply log:
$$\arg\max_\theta \prod_i a_i(\theta) = \arg\max_\theta \sum_i \log a_i(\theta).$$
Then we minimize negative average log-likelihood (NLL). *Insight.* Likelihood rewards parameters that assign mass to what happened; it is silent about what *could* happen unless regularised. *Lineage.* Fisher's likelihood programme; modern empirical risk minimisation inherits the template.

Mental model. log turns "multiply tiny probabilities" into "sum manageable negatives"—mirror of Preface Step 8.

- **Expectation as average under a distribution.** If X is discrete:

$$\mathbb{E}[f(X)] = \sum_x p(x) f(x).$$

This lets us interpret empirical losses as estimates of population quantities. *Insight.* Expectation linearises aggregation—bias/variance decompositions for estimators start here. *Lineage.* Integral formulation ascends to Lebesgue; discrete view suffices for finite vocabularies. *Mental model.* Preface Step 5 bridges to finite-batch stochastic optimisation noise.

- **Entropy, cross-entropy, and KL in one chain.** For distributions p (reference/true) and q (model):

$$H(p) = -\sum_x p(x) \log p(x), \quad H(p, q) = -\sum_x p(x) \log q(x),$$

$$D_{\mathrm{KL}}(p \| q) = \sum_x p(x) \log \frac{p(x)}{q(x)}.$$

Then

$$H(p, q) = H(p) + D_{\mathrm{KL}}(p \| q).$$

Interpretation in coding language: $H(p, q)$ is how many nats you pay on average if you compress draws from p using an optimal code designed for q. Split that bill into two parts: $H(p)$ pays for irreducible uncertainty in p itself; $D_{\mathrm{KL}}(p \| q)$ pays the *extra* bits from model mismatch. Because $H(p)$ does not depend on q, minimising $H(p, q)$ over model parameters is the same optimisation problem as minimising $D_{\mathrm{KL}}(p \| q)$; cross-entropy loss is simply the sample estimate of $H(p, q)$. *Insight.* Optimising CE pushes q toward empirical frequencies while respecting parameterisation constraints. *Lineage.* Shannon (entropy/coding); Kullback–Leibler (1951); revived obsessively in deep learning loss engineering. *Mental model.* KL is asymmetric—treat direction carefully when KL penalties anchor policies or priors.

- **Softmax as probability map from logits:**

$$\mathrm{softmax}(z)_i = \frac{e^{z_i}}{\sum_j e^{z_j}}.$$

Shift invariance (numerical stability):

$$\mathrm{softmax}(z) = \mathrm{softmax}(z - c\mathbf{1}) \quad \text{for any scalar } c.$$

Proof sketch: multiplying every numerator and the normalising denominator by e^{-c} leaves all ratios unchanged. Choosing $c = \max_i z_i$ forces every exponent to be ≤ 0, so each $e^{z_i - c} \in (0, 1]$ and the sum cannot explode upward before you take logs downstream. *Insight.* Softmax is the exponential-family "linear predictor $\to$ simplex" map used in multinomial logistic regression for decades. *Lineage.* Statistical mechanics analogies (Boltzmann/Gibbs); modern LM heads inherit the template unchanged. *Mental model.* Large logits amplify sensitivity—temperature scaling literally rescales that curvature.

- **Perplexity as exponentiated cross-entropy.**

$$\mathrm{PPL} = \exp(H)$$

when H is in nats. Perplexity is interpretable only with fixed tokenization and log base. *Insight.* Perplexity converts cross-entropy into "effective branching factor" language Shannon favoured when estimating English entropy. *Lineage.* Shannon's guessing-game estimates; modern LM tooling reproduces the metaphor for engineers who dislike logs. *Mental model.* Changing tokenizer granularity rescales typical perplexities—never compare runs blindly.

- **Calibration distinction (fit vs honesty of confidence).** Training loss averages $-\log q_\theta(\text{true token})$—it measures whether probability mass sits on what happened in data. *Calibration* asks a different question: when the model assigns predicted probability s to its top choice (or to events it phrases as "confident"), does that choice hit roughly s fraction of the time on held-out slices? A model can achieve moderately good cross-entropy yet assign 0.9 to answers that are only correct 60% of the time (overconfidence), or spread mass flatly and look calibrated while guessing poorly. Always separate "did we assign probability to what occurred?" from "do stated probabilities match empirical frequencies?". *Insight.* Proper scoring rules optimise sharpness *and* honesty under correctly specified beliefs; finite-parameter nets rarely satisfy both simultaneously without explicit calibration machinery. *Lineage.* Brier (1950); modern neural calibration literature (Guo et al., 2017, among others). *Mental model.* Deploy metrics conditional on confidence bins—aggregate accuracy hides structured lies about uncertainty.

How These Results Build on Each Other

1. Conditional probability defines next-token prediction.
2. Chain rule extends next-token predictions to full-sequence probabilities.
3. Likelihood turns full-sequence probabilities into a training objective.
4. Log transform converts objective from product form to additive form.
5. Empirical averaging converts sample objective to dataset objective.
6. Cross-entropy/KL decomposition gives interpretation of what minimizing the objective does.
7. Softmax provides a valid distribution over vocabulary for each context.
8. Calibration analysis audits whether confidence values are trustworthy after fitting.

Reminder Glossary (Term by Term)

- **Random variable:** a map from outcomes to numerical values.
- **Conditional probability:** probability under restricted information.
- **Likelihood:** probability of observed data viewed as function of parameters.
- **Log-likelihood:** logarithm of likelihood; additive over samples/tokens.

- **NLL:** negative log-likelihood; minimized in training.
- **Entropy $H(p)$:** uncertainty of true distribution.
- **Cross-entropy $H(p, q)$:** expected code length when coding p with model q.
- **KL divergence $D_{\mathrm{KL}}(p\|q)$:** mismatch penalty from using q when truth is p.
- **Logits:** pre-softmax scores in $\mathbb{R}^{|V|}$.
- **Softmax:** normalization from logits to a probability simplex.
- **Perplexity:** exponentiated average NLL/cross-entropy (with fixed log base).
- **Calibration:** correspondence between predicted confidence and observed accuracy.
- **ECE:** binned estimate of calibration gap.

Worked Example (Non-Toy, End-to-End Reasoning)

Assume a validation slice with $N = 1000$ predicted tokens. We observe:

- Mean NLL $= 1.10$ nats.
- Mean confidence of top-1 predictions $= 0.78$.
- Actual top-1 accuracy $= 0.64$.

Now reason in the chain used in Chapter 1:

Step 1: Loss to perplexity.

$$\mathrm{PPL} = \exp(1.10) \approx 3.00.$$

Interpretation: average uncertainty corresponds to about three equiprobable choices per token on this slice.

Step 2: Loss does not equal calibration. Despite moderate loss, confidence-accuracy gap is

$$\Delta_{\mathrm{cal}} = 0.78 - 0.64 = 0.14.$$

The model is overconfident by 14 percentage points on average.

Step 3: Why this matters for downstream chapters. Chapter 9 (alignment) can reduce harmful outputs while still leaving calibration issues. Chapter 10 (RAG/evaluation) can improve factual grounding while confidence remains miscalibrated. Therefore the distinction between fit quality and confidence quality is operational, not philosophical.

Step 4: Practical intervention path.

1. Keep objective fit diagnostics (NLL/perplexity).
2. Add calibration diagnostics (reliability bins, ECE).
3. Apply temperature scaling on held-out data.
4. Re-check both loss and calibration after scaling.

Intuition Checkpoints

- A lower NLL does *not* guarantee calibrated confidence.
- KL is a divergence, not a metric; direction matters.
- Perplexity is only comparable under matched tokenization and log base.
- ECE is a binned approximation and depends on binning and sample size.

If You Get Stuck

- Rewrite every probability expression in words before manipulating symbols.
- Track whether each quantity is theoretical, empirical estimate, or model output.
- Check if the reasoning step is algebraic (identity) or statistical (estimation).

References and What To Use Them For

- **Murphy, *Probabilistic Machine Learning*** — conditional probability, expectation, estimation viewpoint.
- **Bishop, *Pattern Recognition and Machine Learning*** — likelihood-based learning, cross-entropy objectives.
- **Cover–Thomas, *Elements of Information Theory*** — entropy, cross-entropy, KL interpretation.
- **Guo et al. (2017)** — practical calibration diagnostics and temperature scaling.
- **Wasserman, *All of Statistics*** — fast refresh on estimation, uncertainty, and empirical risk language.

Chapter 2

Chapter 2 Refresher: Optimization, Generalization, and Scaling Intuition — Optimization and Scaling

SETTING THE SCENE

Chapter 2 consumes the gradient of an empirical risk built in Chapter 1 (Preface Steps 5–8): we optimise an expectation estimated from batches. The novelty is size: billions of parameters force *cheap, noisy* directional probes rather than exact curvature solves.

Step A — Local linear model. Problem. Full loss landscapes are unwieldy; near θ_t we approximate $\mathcal{L}(\theta_t + \Delta)$ by first-order Taylor expansion. **Insight.** Gradient descent assumes meaningful linear prediction within chosen step lengths. **Lineage.** Cauchy (1847) gradient methods; modern proofs invoke convex-like intuitions rarely satisfied literally in deep nets. **Mental model.** If curvature tightens, shrink steps or add momentum filters—otherwise linearisation lies.

Step B — Stochastic gradients. Problem. Exact $\nabla\mathcal{L}$ over full corpora is unaffordable. **Insight.** Subsample to inject noise whose variance trades memory for stability (Preface Step 6). **Lineage.** Robbins–Monro (1951); Bottou's large-scale ML synthesis. **Mental model.** Treat minibatch gradient as true gradient + zero-mean perturbation; diagnose spikes as breakdown of bounded-variance stories.

Step C — Adaptive signal processing. Problem. Parameter blocks receive gradients at wildly different scales (embeddings vs logits vs biases). **Insight.** Running moment estimates rescale updates elementwise while preserving descent intent at smooth regions. **Lineage.** Polyak momentum; Duchi/Hazan/Singer (Adagrad); Kingma/Ba (Adam); Loshchilov/Hutter (AdamW). **Mental model.** Adaptive methods behave like learned preconditioners—trust them until drift/outliers falsify stationarity.

Step D — Numerical hygiene. Problem. Low-precision tensors amplify underflow/overflow. **Insight.** Loss scaling preserves gradient magnitude while exploiting hardware throughput. **Lineage.** NVIDIA mixed-precision training recipes (2017 era onward). **Mental model.** Clip gradients when tail events violate FP16 comfort zones.

Step E — Macroscopic empirical laws. Problem. Engineers must budget parameters *and* tokens jointly. **Insight.** Iso-loss contours summarize decades of runs—Preface Step 7 interpreted informally at massive n. **Lineage.** Kaplan et al. (2020); Hoffmann et al. / Chinchilla (2022). **Mental model.** Scaling fits extrapolate poorly across dataset contamination regimes—treat coefficients as local Taylor surrogates.

Connective thread. Exact gradients are expensive; minibatches trade bias for variance: write $\hat{g}_t = \nabla_\theta \mathcal{L}_t(\theta_t) + \varepsilon_t$ with $\mathbb{E}[\varepsilon_t] = 0$. Under i.i.d. sampling, $\mathrm{Var}(\varepsilon_t)$ scales roughly like $1/B$ with batch size B, which is why larger batches smooth updates but cost memory. Momentum and adaptive moments are filters on that noise; clipping guards rare spikes that violate the "bounded variance" story.

Notation Introduced in This Chapter

- θ: model parameters.
- $\mathcal{L}(\theta)$: objective (loss) as function of parameters.
- $\nabla\mathcal{L}(\theta)$: gradient vector.
- η_t: learning rate at step t.
- $\hat{g}_t$: stochastic (minibatch) gradient estimate.
- m_t, v_t: first and second moment EMAs.
- β_1, β_2: EMA decay coefficients.
- τ: clipping threshold.
- N, D: model/data scale variables used in scaling-law fits.

What You Need To Recall Precisely

- **Gradient as first-order local direction.** If $\mathcal{L}(\theta)$ is differentiable, then for small Δ:

  $$\mathcal{L}(\theta + \Delta) \approx \mathcal{L}(\theta) + \nabla\mathcal{L}(\theta)^\top \Delta.$$

  Choosing $\Delta = -\eta\nabla\mathcal{L}$ gives first-order descent.

- **Stochastic gradient descent (SGD):**

  $$\theta_{t+1} = \theta_t - \eta_t \nabla\mathcal{L}(\theta_t).$$

  In practice, $\nabla\mathcal{L}$ is replaced by minibatch estimate $\hat{g}_t$.

- **Momentum as exponential averaging.** With $m_t = \beta m_{t-1} + (1-\beta)\hat{g}_t$, updates use smoothed direction m_t. Think of m_t as a low-pass filter on the gradient signal: high-frequency minibatch noise is attenuated while consistent downhill curvature survives, similar in spirit to a heavy ball continuing through shallow oscillations in the loss surface.

- **Bias correction for startup EMA.** Because $m_0 = 0$, early EMAs are biased toward zero—the running average "forgets" that gradients were nonzero before time zero. Adam uses debiased estimates

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \qquad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}.$$

  so that if $\hat{g}$ were constant, $\hat{m}_t$ would recover that constant after a few steps instead of ramping up spuriously slowly.

- **AdamW intuition (brief).** Classic Adam entangles weight decay with adaptive preconditioning in ways that distort the intended ℓ_2 penalty. AdamW applies weight decay *directly* to θ (decoupled from the adaptive gradient transform), while still using $\hat{m}_t, \hat{v}_t$ to precondition the loss gradient step.

- **Gradient clipping rule:**

$$g \leftarrow g \cdot \min\left(1, \frac{\tau}{\|g\|}\right).$$

  This bounds update magnitude without changing direction when clipping is active.

- **Mixed precision with loss scaling.** Forward/backward often run in bf16/fp16 for throughput; tiny gradients can flush to subnormals or zero in those formats. Multiply the scalar loss by a large factor s before backward pass so chain-rule gradients inherit scale; divide accumulated gradients by s before the optimizer applies updates (often fp32)—recovering usable magnitude without changing the mathematically correct descent direction.

- **Scaling-law interpretation.** Empirical forms such as

$$L(N, D) = L_\infty + aN^{-\alpha} + bD^{-\beta}$$

  summarise measured iso-loss contours across runs when architecture and data pipelines stay comparable. Use them as *budget planners*: given fixed compute $C \approx 6ND$ tokens-parameter conventions from Kaplan/Chinchilla-style analyses, pick (N, D) pairs predicted to sit near the elbow rather than extrapolate blindly across regimes where data quality or objectives shifted.

- **Constrained optimisation and Lagrange multipliers.**

  Two later chapters (Chapter 6's Chinchilla compute-optimal derivation and Chapter 9's DPO loss derivation) require minimising or maximising a function subject to an equality constraint. The technique is Lagrange multipliers; the argument below is the prerequisite.

  The setup. Minimise $f(\mathbf{x})$ subject to $g(\mathbf{x}) = 0$. At a constrained optimum, the gradient of f must be parallel to the gradient of g (otherwise a feasible step reducing f would exist). Introducing a scalar multiplier λ, the optimum satisfies:

$$\nabla f(\mathbf{x}^\star) = \lambda \nabla g(\mathbf{x}^\star), \qquad g(\mathbf{x}^\star) = 0.$$

  Equivalently, define the Lagrangian $\mathcal{L}(\mathbf{x}, \lambda) = f(\mathbf{x}) - \lambda\, g(\mathbf{x})$ and set $\nabla_{\mathbf{x}} \mathcal{L} = 0$ and $\partial_\lambda \mathcal{L} = 0$.

  Application: Chinchilla compute-optimal allocation (Chapter 6). Fix compute budget $C = 6ND$ (FLOPs, where N = parameters, D = training tokens). The Chinchilla result minimises $L(N, D)$ subject to $6ND = C$. The Lagrangian is $L(N, D) - \lambda(6ND - C)$. Setting partial derivatives to zero and eliminating λ yields the compute-optimal ratio D/N; the resulting allocation is $N^\star \propto C^{1/2}$, $D^\star \propto C^{1/2}$, both scaling as the square root of compute.

- **Normalising constants and partition functions.**

  A second tool required by Chapter 9 is the *normalising constant* (also called the partition function in the statistical-physics framing that RLHF literature inherits).

  The setup. Given an unnormalised non-negative function $\tilde{p}(y)$, the normalised distribution is

$$p(y) = \frac{\tilde{p}(y)}{Z}, \qquad Z = \sum_y \tilde{p}(y).$$

  Z is the partition function. It depends on all outcomes y jointly; in general, computing it requires summing over all possible y, which may be intractable.

  Why it matters for DPO (Chapter 9). The KL-regularised RL optimum has the form

$$\pi^\star(y \mid x) = \frac{\pi_{\text{ref}}(y \mid x)\, e^{r(x,y)/\beta}}{Z(x)}, \qquad Z(x) = \sum_y \pi_{\text{ref}}(y \mid x)\, e^{r(x,y)/\beta}.$$

  $Z(x)$ is intractable to compute directly. The DPO derivation avoids this by taking the *ratio* of $\pi^\star$ at two outputs y^+ and y^-, at which point $Z(x)$ cancels:

$$\frac{\pi^\star(y^+ \mid x)}{\pi^\star(y^- \mid x)} = \frac{\pi_{\text{ref}}(y^+ \mid x)\, e^{r(x,y^+)/\beta}}{\pi_{\text{ref}}(y^- \mid x)\, e^{r(x,y^-)/\beta}}.$$

  The rest of the DPO derivation is algebraic manipulation of this ratio. The partition function cancellation is the key step; understanding that $Z(x)$ is the same scalar in numerator and denominator is all you need.

How These Results Build on Each Other

1. First-order approximation motivates gradient descent.
2. Minibatch estimation introduces noise, requiring variance management.
3. Momentum/adaptive moments stabilize noisy trajectories.
4. Bias correction fixes startup distortion in adaptive moments.
5. Clipping and precision controls handle numerical and tail-event instability.
6. Scaling fits summarize many such runs for planning decisions.
7. Lagrange multipliers convert a constrained minimisation into an unconstrained one, enabling the Chinchilla compute-optimal allocation and the DPO policy derivation.
8. Partition-function cancellation in ratios makes intractable normalising constants disappear, which is the algebraic engine of DPO.

Reminder Glossary (Term by Term)

- **Gradient:** local rate-of-change vector for loss.
- **Learning rate:** step size in parameter space.
- **Minibatch noise:** estimation error from finite sample gradients.
- **Momentum/EMA:** smoothed running estimate of gradient direction.
- **Second moment estimate:** running estimate of squared gradients.
- **Bias correction:** startup debiasing of EMAs.
- **Gradient clipping:** norm-bound safeguard against spikes.
- **Warmup:** gradual LR increase to stabilize early dynamics.
- **Scaling law:** empirical relation among model size, data, compute, and loss.
- **AdamW:** Adam-style adaptive steps with weight decay applied directly to parameters (decoupled from gradient preconditioning).
- **Lagrange multiplier (λ):** scalar that enforces an equality constraint by augmenting the objective; at the constrained optimum, $\nabla f = \lambda \nabla g$.
- **Lagrangian:** the augmented function $\mathcal{L}(\mathbf{x}, \lambda) = f(\mathbf{x}) - \lambda\, g(\mathbf{x})$ whose unconstrained stationary points solve the constrained problem.
- **Partition function (Z):** normalising constant for an unnormalised distribution; often intractable to compute but cancels in ratios of probabilities.

Worked Example (Training-Dynamics Diagnosis)

Suppose a run exhibits:

- training loss oscillation in first 2k steps,
- occasional gradient norm spikes from 12 to 140,
- mixed-precision overflow warnings.

Use the refresher's reasoning chain:

1. **Diagnose instability source.** Early oscillation plus spikes suggests oversized effective step on high-curvature/noisy minibatches.
2. **Apply clipping policy.** With threshold $\tau = 20$, a spike at norm 140 is rescaled by factor $20/140 \approx 0.143$.

3. **Check EMA startup behavior.** For $\beta_1 = 0.9$ and constant scalar gradient 2:

$$m_1 = 0.1 \cdot 2 = 0.2, \quad m_2 = 0.9(0.2) + 0.1(2) = 0.38, \quad m_3 = 0.542.$$

   Bias-corrected:

$$\hat{m}_3 = \frac{m_3}{1 - 0.9^3} = \frac{0.542}{0.271} \approx 2.00.$$

4. **Stabilize numerics.** Increase loss-scaling headroom or use dynamic scaling rollback on overflow.

5. **Interpret outcome.** If oscillation dampens while validation trend improves, intervention matched failure mechanism.

Intuition Checkpoints

- Adam corrections are not magic; under drifting distributions they are approximate.
- Clipping controls step magnitude, not objective geometry.
- Mixed precision saves memory/throughput but shifts numerical failure modes.
- Scaling-law extrapolations degrade when data quality and objective shift.

Pointers

- Loss and KL notation: see Chapter 1 refresher.
- Matrix and norm geometry used by clipping and curvature intuition: see Chapter 4 refresher.

References and What To Use Them For

- **Goodfellow–Bengio–Courville, *Deep Learning*** — SGD, momentum, adaptive optimization intuition.
- **Kingma and Ba (Adam)** — original adaptive-moment method and bias correction rationale.
- **CS229/CS231n notes** — optimization diagnostics and practical training heuristics.
- **Hoffmann et al. (2022)** — compute-optimal scaling interpretation and planning.
- **Bottou et al., optimization for large-scale ML** — stochastic optimization theory background.

Chapter 3

Chapter 3 Refresher: Neural Networks

SETTING THE SCENE

Chapter 3 names the map f_θ that converts contexts (embedded as vectors) into logits consumed by the softmax head from Chapter 1. Probability Preface Steps 2–3 built distributions over *symbols*; here we build differentiable functions producing *parameters* of those distributions.

Step A — Affine versus nonlinear composition. Problem. Stack linear maps naïvely and nothing new appears (matrix multiplication absorbs depth). **Insight.** Insert nonlinearities so each layer genuinely bends space. **Lineage.** McCulloch–Pitts abstract neurons; Rosenblatt perceptron (1958); Minsky–Papert XOR diagnosis (1969). **Mental model.** Ask "would removing φ reduce this entire stack to one matrix?" If yes, redesign.

Step B — Depth becomes tractable. Problem. Naïve depth amplifies or shrinks signals exponentially. **Insight.** Backprop applies Preface Step 8's log-product intuition in reverse: derivatives multiply along layers too. **Lineage.** Werbos thesis (1974); Rumelhart–Hinton–Williams *Nature* (1986). **Mental model.** Forward multiply matrices; backward multiply transposes—same $\mathcal{O}(P)$ order as one forward pass.

Step C — Initialise like a physicist. Problem. Random weights inject variance into activations; depth compounds variance sums (Preface Step 6). **Insight.** Match Var layer-to-layer so neither explosions nor vanishing dominate early training. **Lineage.** Glorot & Bengio (2010); He et al. (2015). **Mental model.** Treat fan-in as degrees of freedom contributing independent squared terms.

Step D — Normalise and short-cut. Problem. Optimisers still distort scale mid-training; identity maps are easiest to learn. **Insight.** Normalisation recentres activations; residuals learn perturbations around identity (He et al., 2016; Ba et al., LayerNorm). **Mental model.** Depth searches neighbourhoods of stable highways rather than rebuilding maps from scratch each layer.

This refresher installs calculus-chain-rule literacy distinct from the probability chain rule (Preface Step 3): same English phrase, different theorem—always label which you mean.

Notation Introduced in This Chapter

- x, h, z: vectors (column vectors by default); $h^{(\ell)}$ activations at layer ℓ.

- $W^{(\ell)}$: weight matrix for layer ℓ; $b^{(\ell)}$: bias vector.
- φ: generic scalar activation applied *elementwise*; φ' its derivative.
- $\sigma(\cdot)$: *only* the logistic sigmoid $\sigma(z) = 1/(1 + e^{-z})$ (overload avoided elsewhere in the reader).
- $\odot$: Hadamard (elementwise) product of two vectors of the same shape.
- $f \circ g$: composition, $(f \circ g)(x) = f(g(x))$.
- J or $\partial u/\partial v$: Jacobian matrix of partial derivatives (shape dictated by numerator layout below).
- $\mathcal{L}$: scalar loss; θ: flattened parameters (as in Chapter 2 refresher).

What You Need To Recall Precisely

- **Affine layer.**
$$z = Wh_{\text{in}} + b, \qquad W \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}},\ h_{\text{in}} \in \mathbb{R}^{d_{\text{in}}},\ b, z \in \mathbb{R}^{d_{\text{out}}}.$$
Without an elementwise nonlinearity, any composition of affine maps is still a single affine map; *depth buys nothing*. Concrete sketch with two layers: $z_2 = W_2(W_1x + b_1) + b_2 = (W_2W_1)x + (W_2b_1 + b_2)$, still affine in x. Adding φ between layers breaks closure because $\varphi(W_1x + b_1)$ is generally non-affine in x.
- **MLP skeleton.** With pre-activations $z^{(\ell)}$, post-activations $h^{(\ell)} = \varphi(z^{(\ell)})$ for hidden layers, and $h^{(0)} = x$,
$$z^{(\ell)} = W^{(\ell)}h^{(\ell-1)} + b^{(\ell)}.$$
The language-modelling head typically applies softmax to logits $z^{(L)}$ (Chapter 1 refresher).
- **Two "chain rules" (disambiguation).**
  - *Probability chain rule* (Chapter 1): factorises a joint as a product of conditionals, e.g. $p(x_{1:T}) = \prod_t p(x_t \mid x_{<t})$.
  - *Calculus chain rule*: propagates derivatives through composed differentiable maps. Same phrase, different theorem.
- **Scalar calculus chain rule.** If $y = g(u)$ and $u = h(x)$ are differentiable scalars, $\dfrac{dy}{dx} = \dfrac{dy}{du}\dfrac{du}{dx}$. For nested $y = f^{(L)} \circ \cdots \circ f^{(1)}(x)$, multiply local derivatives along the graph from output backward.
- **Jacobian view for vectors (reading backprop).** Warmup scalar-on-vector case: if $z \in \mathbb{R}^m$ feeds scalar $\mathcal{L}(z)$ and $z = z(\theta)$, chain rule gives $\partial\mathcal{L}/\partial\theta_j = \sum_i (\partial\mathcal{L}/\partial z_i)(\partial z_i/\partial\theta_j)$—a dot product of the upstream sensitivity $\nabla_z\mathcal{L}$ with each column of $\partial z/\partial\theta$. Packing columns yields matrix-vector form below.

  Take scalar $\mathcal{L}(z)$ with $z \in \mathbb{R}^m$ depending on $\theta \in \mathbb{R}^r$. With *column* gradients $g_z = \nabla_z\mathcal{L} \in \mathbb{R}^m$ and $g_\theta = \nabla_\theta\mathcal{L} \in \mathbb{R}^r$,
$$g_\theta = \Big(\frac{\partial z}{\partial \theta}\Big)^\top g_z,$$
where $\partial z/\partial\theta$ has shape $m \times r$ (row i: $\partial z_i/\partial\theta$). Each layer implements one structured Jacobian–vector product; backprop is the reverse topological walk applying those products without forming full Jacobians explicitly.

- **Elementwise φ and Hadamard.** If $h = \varphi(z)$ elementwise and upstream column gradient is $g_h = \nabla_h \mathcal{L}$, then
$$g_z = g_h \odot \varphi'(z),$$
with $\varphi'(z)$ applied elementwise (convention at nondifferentiable points, e.g. ReLU at 0, fixed for implementations).

- **Affine backward pattern (conceptual).** For $z = Wh + b$, sensitivities satisfy $g_h = W^\top g_z$ and $g_W = g_z\, h^\top$ (up to batch averaging conventions). You do not need to memorise full layouts if you remember: *multiply by W forward, by $W^\top$ backward on activations.*

- **Softmax plus cross-entropy gradient.** Let logits $z \in \mathbb{R}^{|V|}$, probabilities $p = \mathrm{softmax}(z)$, and one-hot target e_k for class/token k. With $\mathcal{L}_{\mathrm{CE}} = -\log p_k$,
$$\nabla_z \mathcal{L}_{\mathrm{CE}} = p - e_k.$$
Sketch: write $\log p_k = z_k - \log \sum_j e^{z_j}$. Differentiate: $\partial(\log \sum_j e^{z_j})/\partial z_i = p_i$ because softmax derivatives telescope inside the log-sum-exp. Subtracting the indicator e_k leaves residual p_i on every coordinate except the target coordinate where it becomes $p_k - 1$. Packaging coordinates yields $p - e_k$. This identity is the bridge between Chapter 1's probabilistic head and Chapter 3's layered map producing logits—gradients vanish exactly when p places mass 1 on the observed token.

- **Hyperplane and XOR.** A linear classifier uses $\mathrm{sign}(w^\top x + b)$; the decision boundary is $\{x : w^\top x + b = 0\}$, an affine hyperplane. XOR labels on $\{(0,0),(0,1),(1,0),(1,1)\}$ are not linearly separable in $\mathbb{R}^2$: no single line separates the two label classes. A hidden layer folds space so a second boundary can finish the job.

- **Variance sketch for initialisation (one layer).** If $z = \sum_{j=1}^{n} w_j x_j$ with w_j independent, zero mean, common variance $\mathrm{Var}(w)$, and fixed x,
$$\mathrm{Var}(z) = \mathrm{Var}(w) \sum_{j=1}^{n} x_j^2.$$
Feeding forward through many layers without controlling $\mathrm{Var}(z)$ leads to exponential growth or shrinkage of activation scale (exploding / vanishing signals). Xavier/He scaling chooses $\mathrm{Var}(w)$ proportional to $1/n_{\mathrm{in}}$ or $2/n_{\mathrm{in}}$ depending on φ so typical squared norms stay stable layer-to-layer—the reader chapter makes this operational.

- **Universal approximation (one sentence).** Wide shallow nets can approximate continuous functions on compact sets under mild conditions on φ; this is about *existence of representation*, not whether SGD finds it quickly or with reasonable width—trainability is separate.

How These Results Build on Each Other

1. Chapter 1 fixes the probabilistic head (softmax, CE); gradients at logits land on $p - e_k$.
2. Chapter 2 consumes $\nabla_\theta \mathcal{L}$ as an abstract vector; Chapter 3 explains how software obtains it through layered Jacobians.
3. Affine maps propagate signals forward (W, b) and sensitivities backward $(W^\top)$.

4. Elementwise φ applies nonlinearity forward and gates gradients backward via $\odot\varphi'(z)$.
5. Composing many layers iterates these two patterns (calculus chain rule), yielding backpropagation when implemented reversibly.
6. Initialisation and normalisation keep forward magnitudes and backward factors in a trainable range; residuals add an identity path so optimisation can make small corrections around a stable baseline.

Reminder Glossary (Term by Term)

- **Pre-activation z:** input to φ before nonlinearity.
- **Post-activation h:** output $\varphi(z)$.
- **Hadamard product $\odot$:** multiply matching coordinates of two vectors.
- **Vanishing gradients:** backward factors shrink exponentially with depth so early layers barely move.
- **Exploding gradients:** backward factors grow until norms overflow numerically.
- **Dead ReLU:** unit stuck with $z \leq 0$ forever receives zero $\varphi'(z)$ and stops learning.
- **BatchNorm / LayerNorm / RMSNorm:** re-centre/rescale activations (reader compares where statistics are taken); purpose includes stabilising scale through depth.
- **Residual / skip connection:** add input of a block to its output so layers learn deltas around identity.

Worked Example (XOR Geometry + One Composition Chain)

Part A (geometry). XOR assigns label $+1$ to $(0,1)$ and $(1,0)$ and label -1 to $(0,0)$ and $(1,1)$. Suppose a linear scoring rule $\text{sign}(w_1x_1 + w_2x_2 + b)$ separates the classes with nonzero margin along each labelled point (ideal separator—relax later). Evaluate linear score $s(x) = w_1x_1 + w_2x_2 + b$ on the four corners. XOR symmetry forces contradictory inequalities:

- Separating $(0,1)$ from $(0,0)$ implies $s(0,1) > s(0,0)$ hence $w_2 > 0$.
- Separating $(1,0)$ from $(0,0)$ implies $w_1 > 0$.
- Separating $(1,1)$ from $(1,0)$ implies $s(1,1) < s(1,0)$ hence $w_2 < 0$.

The second and third requirements contradict each other. No affine hyperplane in $\mathbb{R}^2$ induces opposite signs on exactly those XOR-labelled subsets without contradiction—so linear threshold units alone fail.

Part B (scalar depth intuition). Let $f(x) = \varphi(w_2\,\varphi(w_1x) + b_1) + b_2$ with scalar x and smooth φ. Then

$$\frac{df}{dx} = \varphi'(u_2)\,w_2\,\varphi'(u_1)\,w_1, \quad u_1 = w_1x + b_1,\ u_2 = w_2\varphi(u_1) + b_2.$$

Each layer contributes a factor $\varphi'(u)\,w$; depth multiplies many such factors—either shrink toward 0 or grow past 1 unless φ', weights, and stabilisers are chosen carefully. This scalar picture is the skeleton of vanishing/exploding gradients in vector nets.

Practical reading implication: when Chapter 3 of the reader walks backpropagation, read each layer as supplying one Jacobian–vector product like the factors above; when it discusses Xavier/He, read $\mathrm{Var}(z)$ reasoning like Part B scaled to fan-in n.

Intuition Checkpoints

- Say "probability chain rule" or "calculus chain rule" explicitly when both appear in one sitting.
- Depth without elementwise φ is equivalent to one affine map.
- $\nabla_\theta \mathcal{L}$ from Chapter 2 is the same object software computes by reverse-mode differentiation through the graph defined here.
- $\odot$ with $\varphi'(z)$ is where ReLU sparsity (zeros) creates dead-unit risk.
- Representation theorems do not guarantee cheap optimisation; judge pipelines by trainability evidence, not approximation slogans alone.

Pointers

- Probability, softmax, cross-entropy, logits: see Chapter 1 refresher.
- Gradients as vectors, SGD/Adam, clipping, scaling budgets: see Chapter 2 refresher.
- Embedding geometry and dot products reused inside attention: see Chapter 4 refresher.

References and What To Use Them For

- **Goodfellow–Bengio–Courville, *Deep Learning*** — backpropagation as computational graphs, activations, initialisation practice.
- **Strang, *Linear Algebra*** — matrix multiplication, transpose, norms.
- **Cristianini & Shawe-Taylor (or similar ML math notes)** — Jacobian and gradient layout conventions if confusion persists.
- **Rumelhart–Hinton–Williams (1986)** — historical chain-rule-through-networks narrative.
- **Glorot & Bengio (2010); He et al. (2015)** — variance-preserving initialisation motivations (original papers optional; reader Chapter 3 operationalises scaling with φ).

Chapter 4

Chapter 4 Refresher: Tokenization and Embeddings — From Text to Vectors — Tokenization and Embedding Geometry

SETTING THE SCENE

What mathematical object is a "token" once computation begins? Preface Steps 1–3 treated symbols as draws from discrete supports; here we engineer those supports.

Step A — Segmentation as design. Problem. Raw Unicode is awkward as Ω; engineers carve strings into token sequences $x_{1:T}$. **Insight.** Tokenisation defines which conditional events $p(x_t \mid x_{<t})$ even exist—it relocates probability mass. BPE greedily merges frequent pairs (deterministic, compression-motivated); Unigram-LM assigns a probability to every candidate segmentation and uses EM to prune the vocabulary iteratively (probabilistic, likelihood-motivated). **Lineage.** BPE (Sennrich et al., 2016) inherits Shannon coding intuitions; Unigram-LM (Kudo 2018) frames tokenization as a latent variable model and applies the Expectation–Maximisation principle. **Mental model.** Freeze tokenizer settings before comparing perplexities—otherwise you compare different Ω projections.

Step B — One-hot linear algebra. Problem. Networks ingest vectors, not categorical IDs. **Insight.** Multiplying by a one-hot vector selects an embedding row; training adjusts the entire matrix jointly with downstream logits. **Lineage.** Lookup tables predate deep learning; GPU kernels formalised gathers/scatters. **Mental model.** Think "row pointer," not magical semantics frozen per type.

Step C — Geometry of similarity. Problem. Retrieval and attention ask "which vectors align?" **Insight.** Dot products encode magnitude-sensitive alignment; cosines emphasise direction—same bilinear core, different normalisation story. **Lineage.** Classical IR (Salton); neural embeddings (Mikolov et al.). **Mental model.** Contextualisation breaks static-vector analogies—nearest neighbours shift with sentence neighbourhood.

Representation therefore choreographs every later dot product; mismatch here propagates like conditioning mismatch in Preface Step 2.

Notation Introduced in This Chapter

- V: tokenizer vocabulary (finite symbol set).
- $\iota(\cdot)$: tokenizer mapping from text to token IDs.
- onehot(i): basis vector for token index i.
- E: embedding matrix.
- e_i: embedding vector for token i.
- u, v, q, d: generic vectors used in similarity comparisons.
- $p(t)$: Unigram-LM unigram probability of token $t \in V$.
- $S(X)$: set of all valid segmentations of a string X from the current vocabulary.
- $p(\mathbf{x})$: probability of a segmentation $\mathbf{x} = (t_1, \ldots, t_k)$, equal to $\prod_i p(t_i)$.
- $c(t)$: expected count of token t across the training corpus under the current probability model.

What You Need To Recall Precisely

- **Tokenizer as function.** A tokenizer defines a mapping from strings to finite sequences of IDs,

$$\iota : \text{text} \mapsto (x_1, \ldots, x_T), \quad x_t \in \{1, \ldots, |V|\}.$$

  Length T varies with string length and merge rules. Every downstream probability is conditioned on this segmentation choice (different tokenisers induce different joint distributions over "the same" Unicode text).

- **One-hot basis and embedding lookup.**

$$e_i = E^\top \text{onehot}(i).$$

  If rows of E index vocabulary elements, multiplying $E^\top$ against the one-hot vector selects row i: e_i is the i-th embedding row treated as a column vector. Implementations skip sparse multiplication entirely and index directly into an embedding table.

- **Segmentation criteria.** BPE, WordPiece, and Unigram optimize different objectives (merge frequency, likelihood-style score, probabilistic segmentation).

- **Unigram-LM tokenizer: EM algorithm sketch.**

  The Unigram-LM tokenizer (Kudo 2018, used in SentencePiece and LLaMA-family models) treats segmentation as a latent variable: the same string X can be split in many ways, and each segmentation $\mathbf{x} = (t_1, \ldots, t_k)$ has probability

$$p(\mathbf{x}) = \prod_{i=1}^{k} p(t_i).$$

The marginal probability of the string X sums over all valid segmentations:

$$p(X) = \sum_{\mathbf{x} \in S(X)} p(\mathbf{x}).$$

Training maximises $\sum_X \log p(X)$ over a large vocabulary initialised from all substrings; the parameters are the unigram probabilities $\{p(t)\}$.

E-step (Viterbi segmentation). For each training string, compute the most probable segmentation using the Viterbi algorithm (dynamic programming over character positions). This gives a set of "responsible" segmentations from which expected token counts $c(t)$ are accumulated:

$$c(t) = \sum_X \frac{(\text{expected uses of } t \text{ in } X)}{1}.$$

In practice, the single most probable segmentation (Viterbi decode) is used as a hard assignment rather than soft expectations, making the step tractable.

M-step (re-estimation). Given counts $c(t)$, normalise to obtain updated unigram probabilities:

$$p(t) \leftarrow \frac{c(t)}{\sum_{t'} c(t')}.$$

Pruning step. After each M-step, tokens whose removal decreases the total corpus log-likelihood by less than a threshold η are pruned from the vocabulary. This iteratively shrinks the vocabulary to the target size $|V|$.

Why this matters for the reader. The SentencePiece library (default tokenizer for many open-weight models) can run Unigram-LM mode. Understanding the EM framing explains: (a) why the same string may have multiple valid tokenisations (used during training via subword regularisation); (b) why the vocabulary selection is likelihood-based rather than frequency-based; and (c) why adding private-corpus tokens requires re-running or initialising the EM loop, not just appending to the merge table as in BPE.

- **Similarity metrics.**

$$u^\top v \quad \text{vs} \quad \cos(u, v) = \frac{u^\top v}{\|u\|\|v\|}.$$

Dot product includes magnitude effects; cosine removes scale.

- **Anisotropy and concentration.** After normalisation, many contextual vectors occupy a thin cone in $\mathbb{R}^d$: cosine similarities become systematically high even between moderately unrelated texts. Retrieval pipelines must therefore distinguish "similar because topical" from "similar because geometry concentrates"—often via calibrated thresholds, hybrid lexical retrieval, or reranking.

How These Results Build on Each Other

1. Tokenization defines discrete symbols; BPE via greedy merges, Unigram-LM via EM on a probabilistic segmentation model.
2. Unigram-LM iterates: E-step (Viterbi best segmentation) → M-step (re-estimate probabilities) → pruning (shrink vocabulary).

3. Symbol IDs become one-hot basis vectors.
4. Embedding matrix maps one-hot vectors into continuous space.
5. Similarity metric defines neighborhood structure.
6. Neighborhood structure influences retrieval and downstream context selection.

Reminder Glossary (Term by Term)

- **Vocabulary:** finite symbol inventory used by the model.
- **Token ID:** integer index assigned by tokenizer.
- **One-hot vector:** canonical basis vector for a token ID.
- **Embedding matrix:** learned lookup table from IDs to vectors.
- **Dot product:** magnitude-sensitive alignment score.
- **Cosine similarity:** direction-only alignment score.
- **Anisotropy:** directional imbalance in vector space.
- **Unigram-LM tokenizer:** tokenizer algorithm that maximises marginal log-likelihood over all segmentations via EM; used in SentencePiece (LLaMA, T5, Gemma).
- **E-step (tokenization):** Viterbi decoding to find the most probable segmentation under current $p(t)$ estimates.
- **M-step (tokenization):** normalise accumulated token counts to obtain new $p(t)$ probabilities.
- **Pruning (tokenization):** iterative vocabulary reduction by removing tokens whose absence causes the smallest log-likelihood drop.
- **Subword regularisation:** training with sampled (non-Viterbi) segmentations to expose the model to ambiguous boundaries, improving robustness.
- **Viterbi algorithm:** dynamic programming algorithm for finding the most probable path through a sequence model in $O(T \cdot |V|)$ time.

Worked Mini Example (Unigram-LM EM Iteration)

Suppose the training corpus is the single string "`low`" with vocabulary $V = \{\texttt{l}, \texttt{o}, \texttt{w}, \texttt{lo}, \texttt{ow}, \texttt{low}\}$ and initial uniform probabilities $p(t) = 1/6$ for all t.

E-step. The valid segmentations of "`low`" and their probabilities are:

$$\underbrace{(\texttt{l}, \texttt{o}, \texttt{w})}_{(1/6)^3 \approx 0.0046} \quad , \quad \underbrace{(\texttt{lo}, \texttt{w})}_{(1/6)^2 \approx 0.028} \quad , \quad \underbrace{(\texttt{l}, \texttt{ow})}_{(1/6)^2 \approx 0.028} \quad , \quad \underbrace{(\texttt{low})}_{1/6 \approx 0.167} \quad .$$

Viterbi picks (`low`) as most probable. Hard-assign counts: $c(\texttt{low}) = 1$, all others 0.

M-step. Normalise: $p(\texttt{low}) \leftarrow 1$, all other probabilities $\leftarrow 0$.

Pruning. Tokens not used in any Viterbi segmentation contribute zero to corpus likelihood; prune them. Vocabulary collapses to {`low`}.

Key takeaway: with a single training example, the algorithm converges to the single-token vocabulary in one iteration. In practice, diverse corpora prevent this collapse, but the mechanics are identical.

Worked Example (Why Metric Choice Changes Outcomes)

Let query vector be $q = (10, 0)$ and two candidates:

$$d_1 = (1, 0), \quad d_2 = (1, 1).$$

Dot product ranking:

$$q^\top d_1 = 10, \quad q^\top d_2 = 10.$$

Tie under dot product. Cosine ranking:

$$\cos(q, d_1) = 1, \quad \cos(q, d_2) = \frac{1}{\sqrt{2}} \approx 0.707.$$

Now d_1 is clearly preferred directionally. Reasoning implication: if your retrieval objective is semantic direction, cosine normalization is often essential; if magnitude encodes useful confidence/strength, dot-product behavior may be intentional.

Intuition Checkpoints

- Keep sequence-position indices separate from vocabulary indices.
- Tokenization choice changes sequence length and therefore perplexity comparability.
- Static-word-vector intuitions transfer only partially to contextual embeddings.
- BPE and Unigram-LM produce different vocabularies for the same corpus; neither is universally better — BPE is deterministic and fast; Unigram-LM is probabilistic and supports subword regularisation.
- The Viterbi algorithm is not specific to tokenization; it is a general DP trick for argmax over sequential latent structure. Seeing it here prepares you for HMM and CRF contexts.
- Unigram-LM pruning is irreversible: once a token is removed from the vocabulary, it cannot reappear. This means private-corpus token extension requires restarting or fine-tuning the full EM procedure, not appending merge rules.

Pointers

- Probability/perplexity interpretation: see Chapter 1 refresher.

References and What To Use Them For

- **Jurafsky–Martin, *SLP*** — tokenizer design and language representation.
- **Strang, *Linear Algebra*** — vector spaces, norms, geometry.
- **Mikolov et al.** — distributed embedding intuition.
- **CS224N materials** — modern embedding/retrieval implementation context.

Chapter 5

Chapter 5 Refresher: The Transformer — Attention as the Core Computational Primitive — Attention, Shapes, and Complexity

SETTING THE SCENE

Attention replaces recurrence's sequential bottleneck with explicit pairwise comparisons implemented as batched matrix multiply—linear algebra becomes architecture instead of notation garnish. But attention alone is position-blind: the formula $\text{softmax}(QK^\top/\sqrt{d_k})V$ treats all positions identically unless position is injected explicitly. This chapter covers both the attention mechanism and the positional encoding family that completes it.

Step A — bilinear compatibility. **Problem.** Position t must aggregate information from other positions weighted by relevance. **Insight.** Dot products score compatibility between transformed representations (queries vs keys). **Lineage.** Bahdanau-style attention (2015); scaled dot-product formulation crystallised in Vaswani et al. (2017). **Mental model.** Think "matching embeddings," not mysticism—everything reduces to tensor contractions.

Step B — softmax as differentiable selector. **Problem.** Hard argmax blocks gradients; we need smooth convex combinations. **Insight.** Softmax (Chapter 1 tools) turns logits into mixing weights—Preface Step 10 intuition specialised row-wise. **Lineage.** Statistical mechanics exponential tilts → neural attention gates. **Mental model.** Temperature inherited from decoding chapters scales sharpness identically in spirit.

Step C — masks encode causal laws. **Problem.** Autoregressive generation forbids peeking rightward. **Insight.** Additive $-\infty$ constraints zero softmax mass—enforce equality constraints algebraically before nonlinear squash. **Lineage.** Decoder masking folklore codified in Transformer stacks. **Mental model.** Same trick handles padding tokens—different semantics, identical mathematics.

Step D — quadratic surface area. **Problem.** Pairwise attention allocates $T \times T$ logits per layer. **Insight.** Long contexts explode memory/compute—motivates FlashAttention-style engineering outside this refresher's scope but inside systems reasoning. **Lineage.** Dao et al. IO-aware kernels

(2022 era). **Mental model.** Preface Step 6 variance arguments resurface when chunking randomises attention pathways.

Step E — injecting position. Problem. Attention is permutation-equivariant; without positional information "the cat sat on the mat" and "the mat sat on the cat" produce identical representations. **Insight.** Add a position-dependent signal to each token representation before or inside attention. Three families appear in the main text: absolute sinusoidal (Vaswani), relative bias (ALiBi), and rotation-based (RoPE). **Lineage.** Vaswani (2017) sinusoids → Shaw et al. relative encodings (2018) → Su et al. RoPE (2021) → Press et al. ALiBi (2022). **Mental model.** RoPE rotates each query and key by an angle proportional to its position; relative position appears as the *difference* of angles, which survives the dot product.

If shapes align, attention is honest matrix cookbookery—when shapes drift, bugs hide silently.

Notation Introduced in This Chapter

- T: sequence length.
- d_{model}: model hidden width.
- H: number of attention heads.
- d_k: per-head key/query dimension.
- Q, K, V: query/key/value matrices.
- M: additive mask matrix applied before softmax.
- $\mathbf{R}_\theta$: 2×2 rotation matrix by angle θ.
- m, n: token position indices; θ_i: per-dimension base frequency in RoPE.

What You Need To Recall Precisely

- **Scaled attention:**
$$\text{Attn}(Q, K, V) = \text{softmax}\left(\frac{QK^\top + M}{\sqrt{d_k}}\right) V.$$
Read row-wise: row t of $QK^\top$ lists dot-product similarities between position-t query vectors and every key vector; softmax turns those logits into nonnegative weights summing to 1; multiplying those weights into V mixes value vectors—attention outputs are convex combinations of values attended-to.

  Why divide by $\sqrt{d_k}$? If components of queries and keys are $\mathcal{O}(1)$ with roughly independent variation across d_k dimensions, dot-product magnitudes grow $\Theta(\sqrt{d_k})$ in expectation (length scales like Euclidean norm). Without scaling, softmax logits become sharply peaked as width grows, driving gradients toward sparse argmax-like behaviour and harming trainability. Rescaling keeps typical logits $\mathcal{O}(1)$ across widths.

- **Mask semantics:** causal mask forbids looking right; padding mask blocks non-content locations.

- **Head decomposition:** independent projections create multiple interaction subspaces.
- **Residual and normalization:** residual streams pass representations forward as $x \leftarrow x + \text{Block}(x)$, letting blocks learn corrections around identity rather than full maps from scratch. Normalisation (LayerNorm/RMSNorm inside Transformer blocks) recentres/rescales activations per token or position so dot products do not drift orders of magnitude mid-training—pairs with residual paths to control gradient scale through depth.
- **Complexity source:** score matrix is $T \times T$, so interaction cost grows with sequence length.
- **Positional encodings — three families.**

  Sinusoidal (absolute). Add a fixed vector $\mathbf{p}_t \in \mathbb{R}^{d_{\text{model}}}$ to each token embedding before the first layer:

$$p_{t,2i} = \sin\left(\frac{t}{10000^{2i/d_{\text{model}}}}\right), \quad p_{t,2i+1} = \cos\left(\frac{t}{10000^{2i/d_{\text{model}}}}\right).$$

  Different dimensions oscillate at different frequencies, giving each position a unique fingerprint. The key property: $\mathbf{p}_t \cdot \mathbf{p}_s$ depends only on the offset $t - s$, so relative distance is encoded implicitly.

  ALiBi (relative bias). Instead of modifying embeddings, subtract a head-specific linear bias from attention logits before softmax:

$$\text{score}_{t,s} = \frac{\mathbf{q}_t \cdot \mathbf{k}_s}{\sqrt{d_k}} - m \cdot |t - s|,$$

  where $m > 0$ is a per-head slope. Distant tokens receive larger penalties; the model learns to prefer local context without any positional vector in the embedding.

  RoPE (rotary position embedding). Encode position by *rotating* query and key vectors before the dot product. For a pair of scalar dimensions (x_{2i}, x_{2i+1}) at position t, apply a 2D rotation by angle $t\,\theta_i$:

$$\mathbf{R}_{t\theta_i} = \begin{pmatrix} \cos t\theta_i & -\sin t\theta_i \\ \sin t\theta_i & \cos t\theta_i \end{pmatrix}, \qquad \theta_i = 10000^{-2i/d_k}.$$

  The critical property follows from the rotation group: for any two positions m and n,

$$(\mathbf{R}_{m\theta_i}\,\mathbf{x})^\top (\mathbf{R}_{n\theta_i}\,\mathbf{y}) = \mathbf{x}^\top \mathbf{R}_{(n-m)\theta_i}\,\mathbf{y}.$$

  The dot product of a rotated query at position m with a rotated key at position n depends only on the *relative* offset $n - m$, not on the absolute positions. This is the algebraic reason RoPE gives length-generalisation properties that absolute encodings lack.

  Why this matters for Chapter 5. The reader derives all three families. RoPE is the default in Llama, Mistral, and most open-weight models released since 2023; understanding the rotation-matrix identity is the prerequisite for following that derivation.

How These Results Build on Each Other

1. Project input vectors into Q, K, V.
2. Form pairwise compatibility scores via $QK^\top$.

3. Apply structural constraints with mask M.
4. Normalize row-wise with softmax.
5. Aggregate values and return contextualized representations.

Reminder Glossary (Term by Term)

- **Query:** representation of what current position is asking for.
- **Key:** representation of what each position offers.
- **Value:** content payload mixed by attention weights.
- **Mask:** hard structural restriction on allowable dependencies.
- **Head:** one parallel attention subspace.
- **KV cache:** stored keys/values for efficient autoregressive decoding.
- **Rotation matrix $\mathbf{R}_\theta$:** a 2×2 matrix that rotates a 2D vector by angle θ; its transpose is its inverse ($\mathbf{R}_\theta^\top = \mathbf{R}_{-\theta}$), so $\mathbf{R}_\theta^\top \mathbf{R}_\phi = \mathbf{R}_{\phi-\theta}$.
- **RoPE:** rotary position embedding; encodes absolute position as a rotation so that dot products yield relative-position information.
- **ALiBi:** attention with linear biases; penalises long-range attention logits with a linear distance term, no positional vectors needed.

Worked Mini Example (Shape Sanity)

Let $T = 4$, $d_{\text{model}} = 8$, heads $H = 2$, so $d_k = 4$.

$$Q, K, V \in \mathbb{R}^{4\times 4} \text{ per head}, \quad QK^\top \in \mathbb{R}^{4\times 4}.$$

After softmax over rows, multiply by V to return $\mathbb{R}^{4\times 4}$ per head, concatenate to $\mathbb{R}^{4\times 8}$.

Worked Example (Reasoning Chain with Causal Mask)

For token position $t = 3$ in a length-4 causal sequence, allowed keys are positions $\{1, 2, 3\}$ only. In row 3 of $QK^\top$, mask sets column 4 score to $-\infty$ before softmax. Result: attention weight at $(3, 4)$ is exactly 0. Interpretation: causality is enforced at score level, not corrected later at output level.

Worked Mini Example (RoPE relative-position identity)

Let $d_k = 2$ (one dimension pair), positions $m = 1$, $n = 3$, angle $\theta = \pi/6$.

$$\mathbf{R}_{m\theta} = \mathbf{R}_{\pi/6} = \begin{pmatrix} \cos(\pi/6) & -\sin(\pi/6) \\ \sin(\pi/6) & \cos(\pi/6) \end{pmatrix} = \begin{pmatrix} \sqrt{3}/2 & -1/2 \\ 1/2 & \sqrt{3}/2 \end{pmatrix}.$$

$$\mathbf{R}_{n\theta} = \mathbf{R}_{\pi/2} = \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix}.$$

For arbitrary $\mathbf{x}, \mathbf{y} \in \mathbb{R}^2$:

$$(\mathbf{R}_{\pi/6}\,\mathbf{x})^\top (\mathbf{R}_{\pi/2}\,\mathbf{y}) = \mathbf{x}^\top \mathbf{R}_{\pi/6}^\top \mathbf{R}_{\pi/2}\,\mathbf{y} = \mathbf{x}^\top \mathbf{R}_{(\pi/2-\pi/6)}\,\mathbf{y} = \mathbf{x}^\top \mathbf{R}_{\pi/3}\,\mathbf{y}.$$

The result depends only on the offset $n - m = 2$ (encoded via angle $\pi/3 = 2 \times \pi/6$), not on the absolute positions 1 and 3. This is the key property in full d_k-dimensional RoPE: decompose into $d_k/2$ independent 2D rotation pairs, apply the same argument in each pair.

Intuition Checkpoints

- "Masking" is a constraint on attention scores, not a post-hoc output edit.
- Quadratic attention cost refers to score interactions, not all block components equally.
- KV cache changes decoding-time behavior, not pretraining-time complexity class.
- RoPE encodes absolute position but the dot product reads only relative position — the rotation group guarantees this algebraically; it is not an approximation.
- ALiBi requires no positional vectors and generalises well to lengths longer than seen in training; sinusoidal PE does not generalise as cleanly beyond the training context window.

Further Refresh

- Shape and vector geometry reminders: see Chapter 4 refresher.
- Softmax stability reminders: see Chapter 1 refresher.
- **Vaswani et al. (2017)** — architectural equations and design rationale.
- **Annotated Transformer** — equation-by-equation implementation mapping.

Chapter 6

Chapter 6 Refresher: Pretraining, GPT-Style Models, and In-Context Learning — Pretraining Objective and Decoding Controls

SETTING THE SCENE

Training optimises Preface Steps 8–10 on teacher-forced contexts; deployment samples from the learned conditional family under budgets and safety envelopes—same θ, different *pushforward* map from logits to tokens.

Step A — Autoregressive likelihood reminder. Problem. Guarantee coherence across positions while fitting data. **Insight.** Factorisation $\prod_t p_\theta(x_t \mid x_{<t})$ ties optimisation to causal graphs already rehearsed in Preface Step 3. **Lineage.** Shannon's entropy estimates for English → modern causal transformers. **Mental model.** Teacher forcing mismatches free-running drift—monitor both regimes.

Step B — Inference-time reshaping. Problem. Raw softmax may be too peaked or too flat for products. **Insight.** Temperature, top-k, top-p apply measure transforms *after* forward pass—no gradient necessarily flows (pure inference policy). **Lineage.** Nucleus sampling (Holtzman et al.) formalises mass truncation psychology. **Mental model.** Treat decoding knobs as UI levers on Shannon entropy (Preface Step 9).

Step C — Scaling budgets. Problem. Engineers trade parameters vs tokens under compute roofs. **Insight.** Empirical iso-loss laws operationalise Preface Step 7 qualitatively at trillion-token scales. **Lineage.** Kaplan (2020); Chinchilla (2022). **Mental model.** Laws fit historical pipelines—extrapolate cautiously when data composition shifts.

Decoding is theatre lighting on the same actor (θ); training chooses the script likelihood, decoding chooses spotlight intensity.

Notation Introduced in This Chapter

- z_i: logit for candidate token i.

- p_i: probability of candidate token i after normalization.
- τ: temperature parameter.
- k: top-k truncation count.
- p: top-p cumulative mass threshold.
- N, D: model/data scale variables in scaling discussions.

What You Need To Recall Precisely

- **Autoregressive NLL objective.** For corpus sequences $x_{1:T}$, training maximises $\sum_t \log p_\theta(x_t \mid x_{<t})$, equivalently minimises per-token cross-entropy averaged over positions and documents. This is the Chapter 1 chain rule factorisation evaluated inside the transformer stack that produces conditional softmax parameters at each t.
- **Temperature transform:**
$$p_i(\tau) = \frac{\exp(z_i/\tau)}{\sum_j \exp(z_j/\tau)}.$$
- **Top-k and top-p (ordered operations).** Start from full softmax masses p_i over vocabulary.
  1. **Top-k:** keep the k largest probabilities, zero the rest, divide remaining masses by their sum so they again form a distribution on the surviving support.
  2. **Top-p (nucleus):** sort probabilities descending, greedily add tokens until cumulative mass reaches threshold $p \in (0, 1]$, zero outside that prefix, renormalise on the kept set.

  Both policies intentionally discard probability mass outside a subset of vocabulary entries before sampling; they reshape diversity without changing trained weights.
- **Objective/metric distinction:** training objective may not align perfectly with deployment metric.
- **Scaling fits:** compute/data/parameter fronts are empirical and objective-dependent.

Pointers

- CE/KL/perplexity: see Chapter 1 refresher.
- Optimizer and scaling-law mechanics: see Chapter 2 refresher.
- Attention architecture assumptions: see Chapter 5 refresher.

How These Results Build on Each Other

1. Train model to minimize autoregressive NLL.
2. Convert logits to probabilities at inference.

3. Apply decoding transform (temperature/truncation).
4. Sample outputs from transformed distribution.
5. Evaluate behavior with metrics that may differ from training objective.

Reminder Glossary (Term by Term)

- **Autoregressive objective:** predict each token conditioned on prefix.
- **Decoding policy:** rule for turning model distribution into emitted tokens.
- **Temperature:** entropy-control knob on logits.
- **Top-k:** fixed-cardinality truncation.
- **Top-p:** mass-based truncation.
- **Scaling law:** empirical relation between loss and resource axes.

Worked Mini Example (Temperature)

Take logits $z = (2, 1, 0)$. With $\tau = 1$:

$$p \approx (0.665, 0.245, 0.090).$$

With $\tau = 2$:

$$p \approx (0.506, 0.307, 0.186).$$

Higher temperature flattens the distribution, increasing diversity and risk simultaneously.

Worked Example (Decode Policy Choice for Same Model)

Suppose next-token distribution after softmax is:

$$(0.52, 0.24, 0.12, 0.07, 0.05).$$

Under top-k with $k = 2$, support becomes first two tokens, renormalized to

$$\left(\frac{0.52}{0.76}, \frac{0.24}{0.76}\right) \approx (0.684, 0.316).$$

Under top-p with $p = 0.9$, first four tokens are retained (mass 0.95), renormalized to:

$$(0.547, 0.253, 0.126, 0.074).$$

Same trained model, different generation behavior due only to policy.

Intuition Checkpoints

- Decoding does not retrain the model; it samples from a transformed output distribution.
- Top-k and top-p can behave similarly in peaked distributions and very differently in flat distributions.
- Apparent "emergence" can be caused by metric discontinuities, not abrupt capability phase transitions.

Further Refresh

- **Hoffmann et al. (2022)** — compute-optimal scaling interpretation.
- **Holtzman et al.** — nucleus sampling and degeneration analysis.
- Standard decoding notes from CS224N/modern LLM systems courses.

Chapter 7

Chapter 7 Refresher: Efficiency, Scaling, and Mixture-of-Experts — Systems Arithmetic and Sparse Routing

SETTING THE SCENE

Once probabilities and networks are defined, training becomes physics on silicon: bytes move, joules dissipate, routers stall. Preface Step 6 (variance of sums) quietly lurks behind contention noise.

Step A — Roofline worldview. Problem. MFLOPS dashboards lie when kernels stream weights constantly. **Insight.** Compare arithmetic intensity (FLOPs/byte) against bandwidth-induced slopes—whichever bound bites first wins. **Lineage.** Williams/Waters/Patterson roofline model (2009) repurposed across ML hardware discourse. **Mental model.** If intensity $<$ ridge crossover, buying more TFLOPs wastes money—buy bandwidth or locality instead.

Step B — Sparsity swaps villains. Problem. Dense matrices saturate compute; sparse routing saves FLOPs but reroutes traffic. **Insight.** MoE trades matmuls for all-to-all collectives—new bottleneck on wires (Preface Step 6 covariance spikes across devices). **Lineage.** Shazeer's Switch Transformer et seq. **Mental model.** Plot effective λ/μ including dispatch latency before celebrating parameter counts.

Step C — Capacity factor as queue abstraction. Problem. Experts overload like servers under burst arrivals. **Insight.** Capacity factor overprovisions slots analogous to multi-server queue stability margins—drops mirror admission control. **Lineage.** Queueing tradition (Kendall notation) meets routing papers. **Mental model.** Tail latency dominates UX—average FLOPs/token misleads.

Systems graphs reinterpret probability routines as throughput limits; migrating bottlenecks obey no moral ranking beyond measurement.

Notation Introduced in This Chapter

- I: arithmetic intensity (FLOPs per byte).
- k: number of selected experts per token in top-k routing.
- N_{active}: active parameters per token.

- N_{total}: total resident parameters.
- CF: capacity factor (expert slot overprovision multiplier).

What You Need To Recall Precisely

- **Bottleneck triad:** compute, memory bandwidth, and interconnect communication.
- **MoE routing:** compute gate logits per token (router network); softmax yields mixture weights; keep top-k experts, zero or mask others, renormalise weights on the surviving experts so each token still mixes convexly over experts it dispatches to; aggregate expert outputs with those weights. Routing induces sparse activation patterns separate from parameter residency.
- **Capacity factor:** finite per-expert token slots induce drop/reroute decisions.
- **Arithmetic intensity:** FLOPs/byte predicts memory-bound vs compute-bound behavior.
- **Active vs total parameters:** sparse models can have large total memory footprint while using small active compute per token.

How These Results Build on Each Other

1. Compute workload demand (FLOPs and bytes/token).
2. Compare with device and interconnect limits.
3. Identify bottleneck (compute, memory, communication).
4. Apply sparse routing and reassess new bottleneck.
5. Validate with throughput/latency measurements.

Reminder Glossary (Term by Term)

- **Arithmetic intensity:** FLOPs per byte transferred.
- **Dispatch/combine:** route tokens to experts and gather outputs.
- **Capacity factor:** overprovisioning multiplier for expert token slots.
- **Token drop:** overflow handling when expert capacity exceeded.
- **All-to-all:** communication primitive central to MoE scaling.

Worked Mini Example (Intensity Intuition)

Assume a kernel performs 2×10^{12} FLOPs and moves 8×10^{11} bytes.

$$I = \frac{2 \times 10^{12}}{8 \times 10^{11}} = 2.5 \text{ FLOPs/byte.}$$

If hardware requires much higher intensity for compute saturation, this kernel is memory-bound.

Worked Example (MoE Bottleneck Migration)

Assume dense model requires 100 units compute and 40 units communication per batch. After MoE sparsification, active compute drops to 45, but dispatch/combine communication rises to 80. System shifts from compute-bound to communication-bound. Interpretation: sparse compute improved one axis but exposed another, so optimization must target network scheduling or expert placement.

Intuition Checkpoints

- Sparse compute does not imply sparse communication.
- MoE can reduce compute/token while increasing routing complexity.
- High model parameter count can coexist with low active compute and high memory residency.

Further Refresh

- Optimization and memory accounting: see Chapter 2 refresher.
- Attention complexity and KV behavior: see Chapter 5 refresher.
- Decoding/token-rate implications: see Chapter 6 refresher.
- **Shazeer et al. / Switch Transformer papers** — sparse routing mechanics and load balancing.
- **Harchol-Balter performance modeling** — bottleneck and throughput reasoning.

Chapter 8

Chapter 8 Refresher: Model Adaptation, LoRA, Quantization, and Deployment Reality — Low-Rank Adaptation and Quantization

SETTING THE SCENE

Deployment freezes most of θ learned via Preface Step 8-style objectives and tunes cheap corrections under storage/power envelopes—approximation theory meets accounting.

Step A — Low-rank hypothesis. Problem. Full matrices encode redundancy; updating all entries per task is wasteful. **Insight.** SVD decomposes any matrix into a sum of rank-1 outer products ordered by singular value magnitude. Restrict ΔW to rank-r manifolds—implicit claim task signals concentrate in few directions (Preface Step 6 thinks about singular values informally). **Lineage.** Eckart–Young (1936) best low-rank approximation theorem; Hu et al. LoRA (2021) popularised modern adapter lore. **Mental model.** Monitor singular spectrum drift when skeptics question rank budgets—a fast-decaying spectrum justifies small r; a flat spectrum does not.

Step B — Quantisation as noisy channel. Problem. Continuous weights exceed SRAM budgets. **Insight.** Rounding injects controlled distortion reminiscent of quantised communication channels—error scales with dynamic range misuse. **Lineage.** Dettmers QLoRA et al. marry low-rank updates with int4 storage. **Mental model.** Outliers behave like heavy-tail Preface Step 6 violations—scale grids adapt or suffer.

Step C — Gate on measured drift. Problem. Approximations trade compute for risk. **Insight.** Acceptance tests slice datasets like calibration bins—aggregate scores hide subgroup collapses tied to Preface Step 5 expectations.

Treat every shortcut as falsifiable: publish metrics or admit folklore.

Notation Introduced in This Chapter

- W: base weight matrix, ΔW: adaptation update.

- A, B: low-rank factors in LoRA update.
- d, k: layer input/output dimensions.
- r: LoRA rank.
- α: LoRA scaling constant.
- ρ: trainable parameter fraction versus full fine-tuning.
- U, Σ, V: left singular vectors, diagonal singular values, right singular vectors in SVD.
- σ_i: the i-th singular value (conventionally $\sigma_1 \geq \sigma_2 \geq \cdots \geq 0$).
- $\|M\|_F$: Frobenius norm, $\sqrt{\sum_{i,j} M_{ij}^2} = \sqrt{\sum_i \sigma_i^2}$.

What You Need To Recall Precisely

- **Singular Value Decomposition (SVD) and the low-rank approximation theorem.**

  Every real matrix $M \in \mathbb{R}^{d\times k}$ factors as

  $$M = U\Sigma V^\top,$$

  where $U \in \mathbb{R}^{d\times d}$ and $V \in \mathbb{R}^{k\times k}$ are orthogonal, and Σ is diagonal with non-negative entries $\sigma_1 \geq \sigma_2 \geq \cdots \geq 0$. Expanding the product gives a sum of rank-1 outer products:

  $$M = \sum_{i=1}^{\min(d,k)} \sigma_i \, \mathbf{u}_i \mathbf{v}_i^\top.$$

  Eckart–Young theorem. For any target rank r, the best rank-r approximation in Frobenius norm (and in spectral norm) is obtained by keeping the top r terms:

  $$M_r = \sum_{i=1}^{r} \sigma_i \, \mathbf{u}_i \mathbf{v}_i^\top, \qquad \|M - M_r\|_F = \sqrt{\sigma_{r+1}^2 + \cdots + \sigma_{\min(d,k)}^2}.$$

  No other rank-r matrix is closer to M in Frobenius norm. This is the theoretical foundation for using low-rank matrices as approximations.

  Connection to LoRA. The LoRA hypothesis is that the task-relevant update ΔW has a rapidly decaying singular spectrum—most of its "signal" lives in a low-dimensional subspace. Restricting $\Delta W = \frac{\alpha}{r} BA$ (with $B \in \mathbb{R}^{d\times r}$, $A \in \mathbb{R}^{r\times k}$) is equivalent to parameterising the rank-r subspace directly. Eckart–Young justifies this when the true update's singular values drop off quickly; when the spectrum is flat, a small r will miss significant directions and underfitting follows.

  Diagnosing rank budget. Inspect the singular value spectrum of ΔW after a full fine-tuning run (if affordable as a baseline). Rapid decay — first few σ_i capturing $\geq 90\%$ of $\|\Delta W\|_F$ — supports a small LoRA rank. Slow decay flags either a hard task shift or a dataset with diverse sub-distributions, both arguing for larger r or continued pretraining.

- **LoRA update form and intuition.**

$$\Delta W = \frac{\alpha}{r} BA, \quad B \in \mathbb{R}^{d\times r},\ A \in \mathbb{R}^{r\times k}.$$

  Rank $r \ll \min(d, k)$ forces ΔW to live in an r-dimensional subspace of the full $d \times k$ parameter space; optimisation restricts plasticity to directions where adaptation concentrates empirically, exchanging expressiveness for parameter savings.

- **Trainable parameter fraction:**

$$\rho = \frac{r(d+k)}{dk}.$$

- **Quantization model:** map tensors through affine rescaling (per-channel or per-tensor scale/zero-point) into finite-bit grids—often asymmetric ranges capture outliers better than naive symmetric buckets. Quantisation noise concentrates where scales mismatch empirical tails (especially outliers), triggering subgroup drift unrelated to aggregate perplexity.
- **Error tradeoff:** memory/bandwidth gains versus approximation and calibration drift risk.
- **Budget accounting:** adaptation and base-model memory are separate components.

How These Results Build on Each Other

1. SVD decomposes any matrix into rank-1 components ordered by importance.
2. Eckart–Young establishes that truncating to rank r is the optimal approximation under Frobenius/spectral norm.
3. The low-rank hypothesis applies this to weight updates: restrict ΔW to rank r if task signal concentrates in few directions.
4. LoRA parameterises the rank-r subspace explicitly with factors B, A.
5. Quantization adds a second approximation layer (bit-width reduction) with independent noise characteristics.
6. Deployment gating measures drift from both approximations before release.

Reminder Glossary (Term by Term)

- **Singular value decomposition (SVD):** factorisation $M = U\Sigma V^\top$ into orthogonal factors and a diagonal matrix of singular values; every real matrix has one.
- **Singular value (σ_i):** non-negative scalar measuring the "size" of the i-th rank-1 component; $\sigma_1 \geq \sigma_2 \geq \cdots$.
- **Singular spectrum:** the sequence $(\sigma_1, \sigma_2, \ldots)$; fast decay supports low-rank approximation, flat spectrum does not.
- **Eckart–Young theorem:** the best rank-r approximation in Frobenius (or spectral) norm is the truncated SVD M_r; there is no better rank-r matrix.

- **Frobenius norm:** $\|M\|_F = \sqrt{\sum_{i,j} M_{ij}^2} = \sqrt{\sum_i \sigma_i^2}$; natural measure of total approximation error.
- **Rank:** dimensionality of adaptation subspace; in LoRA, the number of singular directions parameterised explicitly.
- **LoRA scaling factor:** controls update magnitude relative to rank.
- **Quantizer:** mapping from continuous values to discrete levels.
- **Calibration set:** data used to estimate quantization scales.
- **Drift threshold:** maximum acceptable quality loss for deployment.

Worked Mini Example (SVD and Eckart–Young)

Let $M = \begin{pmatrix} 3 & 0 \\ 0 & 1 \\ 0 & 0 \end{pmatrix}$. The SVD is $M = U\Sigma V^\top$ with

$$U = I_3, \quad \Sigma = \begin{pmatrix} 3 & 0 \\ 0 & 1 \\ 0 & 0 \end{pmatrix}, \quad V = I_2,$$

giving singular values $\sigma_1 = 3$, $\sigma_2 = 1$. The best rank-1 approximation keeps only σ_1:

$$M_1 = \begin{pmatrix} 3 \\ 0 \\ 0 \end{pmatrix} (1\ 0) = \begin{pmatrix} 3 & 0 \\ 0 & 0 \\ 0 & 0 \end{pmatrix}, \qquad \|M - M_1\|_F = \sigma_2 = 1.$$

No other rank-1 matrix achieves Frobenius error below 1. The ratio $\sigma_1^2/(\sigma_1^2 + \sigma_2^2) = 9/10 = 90\%$ of total squared norm is captured by rank 1—a fast-decaying spectrum that supports the low-rank approximation.

Worked Mini Example (Parameter Fraction)

Let $d = k = 4096$ and $r = 16$:

$$\rho = \frac{16(4096 + 4096)}{4096 \cdot 4096} = \frac{131072}{16777216} \approx 0.0078.$$

This is about 0.78% of full fine-tuning parameters for that layer.

Worked Example (Quality Gate Under Approximation)

Suppose full fine-tuned baseline scores 78.4 on target benchmark. LoRA+quantized candidate scores 77.1 overall, but subgroup analysis shows:

- common cases: -0.6
- rare long-context cases: -3.8

Reasoning: average drop may appear acceptable, but concentrated degradation violates deployment intent for long-context tasks. Decision: reject current quantization profile or introduce subgroup-aware gate.

Intuition Checkpoints

- SVD always exists and is unique (up to sign); it is not an approximation — it is an exact factorisation. The approximation step is the truncation to rank r.
- Eckart–Young gives a lower bound on approximation error: $\|M - M_r\|_F \geq \sigma_{r+1}$. No representation trick can do better for a given rank.
- A low-rank update is a hypothesis about task-adaptation dimensionality, not a theorem. Inspect the singular spectrum to validate it.
- Quantization quality is distribution-dependent; outlier channels can dominate failure.
- "Near parity" must be defined metric-by-metric, not as a vague label.

Further Refresh

- Rank and linear-algebra background: see Chapter 4 refresher.
- Optimizer/memory arithmetic: see Chapter 2 refresher.
- Systems bottlenecks and throughput: see Chapter 7 refresher.
- **LoRA (Hu et al.)** — low-rank adaptation mechanics.
- **QLoRA (Dettmers et al.)** — quantization plus adapter training.

Chapter 9

Chapter 9 Refresher: Alignment, Instruction Tuning, and Preference Optimization — Preferences, KL Regularization, and Policy Updates

SETTING THE SCENE

Alignment reframes Preface Steps 2–4: instead of conditioning only on observed tokens, we condition on human comparisons and regularise policies toward trusted references.

Step A — Pairwise likelihood. Problem. Scalar correctness insufficient; humans supply partial orders. **Insight.** Bradley–Terry models lift score gaps into Bernoulli probabilities—Preface Step 2 specialised to preference events. **Lineage.** Thurstone/B Bradley–Terry tradition; Rafailov et al. DPO (2023) streamlines derivatives. **Mental model.** Relative labels estimate latent utilities; absolute calibration remains optional extra.

Step B — KL anchor. Problem. Unconstrained policy optimisation hallucinates unsupported modes. **Insight.** KL penalties replay Preface Step 10 mismatch narrative between π_θ and π_{ref}. **Lineage.** Schulman/PPO lineage intersects RLHF practice (Stiennon, Ouyang, subsequent stacks). **Mental model.** β trades helpfulness vs behavioural drift—no universal optimum without governance context.

Step C — Support hazards. Problem. Zero reference probability yields exploding log-ratios. **Insight.** Conditioning events need positive measure—same vigilance as Preface Step 3 warned for chain factors.

Preference optimisation is probabilistic inference under sociotechnical data—mathematics unchanged, likelihood rewritten.

Notation Introduced in This Chapter

- x: prompt/context.
- y^+, y^-: preferred and dispreferred responses.

- $r(x, y)$: reward/score function.
- $\sigma(\cdot)$: logistic sigmoid function.
- π: current policy distribution.
- π_{ref}: reference policy.
- β: KL regularization coefficient.

What You Need To Recall Precisely

- **Bradley–Terry form:**

$$P(y^+ \succ y^- \mid x) = \sigma(r(x, y^+) - r(x, y^-)).$$

  Write margin $\Delta r = r(x, y^+) - r(x, y^-)$. In Bradley–Terry style models this acts like a log-odds difference: $\sigma(\Delta r)$ matches how humans choose stochastically under utility noise; pairwise losses therefore encourage reward gaps consistent with observed preferences.

- **KL regularization role.** Optimising preferences alone can peel π_θ away from pretrained π_{ref} into unsupported linguistic regimes where rewards mis-generalise. Adding $\beta\, D_{\text{KL}}(\pi_\theta \,\|\, \pi_{\text{ref}})$ (or sampled surrogates thereof) penalises drift measured against data-supported behaviour while pairwise margins tilt π_θ toward helpful completions.
- **Pairwise objective geometry:** score differences (margins) drive learning signal.
- **Support condition:** near-zero reference probability destabilizes log-ratio terms.
- **Objective-vs-behavior distinction:** optimizing preference loss does not automatically guarantee safety or honesty on all distributions.

How These Results Build on Each Other

1. Convert pairwise labels into likelihood terms.
2. Learn scoring or policy update from pairwise margins.
3. Add KL regularization to bound policy drift.
4. Validate on held-out safety/helpfulness metrics.

Reminder Glossary (Term by Term)

- **Preference pair:** chosen vs rejected response for same prompt.
- **Margin:** score difference between preferred and non-preferred outputs.
- **KL penalty:** regularizer limiting divergence from reference policy.
- **Support mismatch:** missing probability mass causing unstable ratios.
- **Over-optimization:** objective improves while true quality degrades.

Worked Mini Example (Pairwise Probability)

If $r(x, y^+) = 2.0$ and $r(x, y^-) = 0.5$:

$$P(y^+ \succ y^- | x) = \sigma(1.5) \approx 0.817.$$

Interpretation: the model assigns roughly 82% preference probability to y^+ under this reward gap.

Worked Example (Preference Gain vs Safety Drift)

Suppose after tuning:

- preference win-rate improves from 62% to 73%,
- refusal-on-unsafe-prompts worsens from 91% to 84%.

Reasoning: preference objective improved but safety behavior regressed. Implication: keep preference metric and safety metric as separate acceptance gates; one does not substitute for the other.

Intuition Checkpoints

- Lower KL does not mean better helpfulness; it means less policy drift.
- Better reward-model fit does not guarantee lower downstream harm.
- Pairwise preferences encode relative order, not absolute truth.

Further Refresh

- KL/cross-entropy foundations: see Chapter 1 refresher.
- Optimization mechanics: see Chapter 2 refresher.
- Adaptation substrate (LoRA/QLoRA): see Chapter 8 refresher.
- **Christiano/Stiennon/Ouyang papers** — RLHF workflow and assumptions.
- **Rafailov et al. (DPO)** — direct preference objective derivation.

Chapter 10

Chapter 10 Refresher: LLM Systems — Retrieval, Tool Use, and Evaluation Harnesses — Retrieval Metrics and Evaluation Arithmetic

SETTING THE SCENE

Retrieval-augmented answering concatenates two stochastic modules: recall relevant evidence (set-valued event), then condition generation on that evidence (Preface Step 2 again). One scalar cannot isolate which module failed.

Step A — Set-valued relevance. Problem. Truth may admit multiple supporting documents. **Insight.** Precision/recall generalise Preface Step 1 counting logic to partial overlaps between retrieved and gold sets. **Lineage.** Classical IR (Manning et al.); neural hybrids layer embedding geometry from Chapter 4.

Step B — Conditional generation quality. Problem. Good retriever + weak synthesiser still ships nonsense. **Insight.** Evaluate $\mathcal{L}$ or task metrics *given* oracle contexts vs live contexts—difference isolates grounding gaps.

Step C — Faithfulness as constrained decoding. Problem. Fluent generators invent citations. **Insight.** Faithfulness metrics approximate hard logical entailment with softer overlap checks—engineering surrogate for impossible full semantics.

Step D — Sparse retrieval scoring and hybrid fusion. Problem. Dense retrievers miss exact-match keywords; sparse retrievers miss paraphrases. Neither alone suffices for private or technical corpora. **Insight.** BM25 is a principled lexical scorer with IDF weighting, TF saturation, and length normalisation. Reciprocal Rank Fusion (RRF) combines BM25 and dense rankings without needing calibrated scores—just ordinal ranks. **Lineage.** BM25 (Robertson & Walker, 1994 BM25 refinements); RRF (Cormack et al. 2009). **Mental model.** RRF's $k = 60$ offset prevents a rank-1 result from dominating; both retrievers contribute robustly even when their score scales are incomparable.

Decomposition honors Preface Step 5 linearity instinct: diagnose components before averaging them blindly.

Notation Introduced in This Chapter

- k: retrieval cutoff (also used as the RRF offset constant, conventionally 60; context distinguishes the two).
- R_k: top-k retrieved set.
- G: gold relevant set.
- Precision@k, Recall@k: standard ranking metrics.
- q: query embedding/vector; also used as query term string.
- d/c: document/chunk embedding/vector.
- t: a query term.
- $\mathrm{TF}(t, c)$: term frequency of t in chunk c.
- $\mathrm{IDF}(t)$: inverse document frequency of t.
- N: corpus size (number of documents).
- $\mathrm{df}(t)$: number of documents containing term t.
- $|c|$: length of chunk c in tokens; avgdl: corpus average chunk length.
- k_1, b: BM25 saturation and length-normalisation parameters (typical: $k_1 \in [1.2, 2.0]$, $b = 0.75$).
- $r_{\mathrm{bm25}}(c, q)$, $r_{\mathrm{dense}}(c, q)$: rank of chunk c in BM25 and dense retrievals for query q.

What You Need To Recall Precisely

- **Ranking metrics.** Let R_k be top-k retrieved IDs and G gold relevant IDs with $|G|$ possibly larger than k.

$$\mathrm{Prec@}k = \frac{|R_k \cap G|}{k}, \qquad \mathrm{Rec@}k = \frac{|R_k \cap G|}{|G|}.$$

  Precision stresses cleanliness of short rankings; recall stresses coverage of all positives—trade-offs arise whenever $|G|$ is large relative to k. Mean reciprocal rank (MRR) averages 1/rank of first relevant across queries—rewarding systems that surface *one* correct doc quickly rather than maximising exhaustive recall.

- **BM25 scoring formula.** For a query q and chunk c, BM25 sums over query terms $t \in q$:

$$\mathrm{BM25}(c, q) = \sum_{t \in q} \mathrm{IDF}(t) \cdot \frac{\mathrm{TF}(t, c) \cdot (k_1 + 1)}{\mathrm{TF}(t, c) + k_1 \left(1 - b + b \cdot \dfrac{|c|}{\mathrm{avgdl}}\right)}.$$

IDF term. Measures how rare term t is across the corpus:

$$\mathrm{IDF}(t) = \log\left(\frac{N - \mathrm{df}(t) + 0.5}{\mathrm{df}(t) + 0.5} + 1\right).$$

Rare terms get high IDF; stop-words appearing in almost every document get IDF near zero.

TF saturation. Raw term frequency is not linear: a document with 10 occurrences is not 10× more relevant than one with 1. The numerator $\mathrm{TF}(t,c) \cdot (k_1 + 1)$ divided by $\mathrm{TF}(t,c) + k_1$ is a saturating function that asymptotes at $(k_1 + 1)$ as $\mathrm{TF} \to \infty$; k_1 controls how fast saturation sets in.

Length normalisation. Long documents contain more terms by chance. The $(1 - b + b \cdot |c|/\mathrm{avgdl})$ denominator term penalises long documents proportionally, with $b = 0$ disabling normalisation entirely and $b = 1$ fully correcting for length.

Why it matters. BM25 is the standard baseline for lexical retrieval in every RAG pipeline. Private corpora with specialised terminology (medical, legal, engineering) benefit from BM25's IDF weighting, which down-weights corpus-wide common terms and up-weights rare domain terms without any learned representation.

- **Reciprocal Rank Fusion (RRF).** Given a BM25 ranked list and a dense-embedding ranked list, the hybrid score for chunk c against query q is:

$$\mathrm{RRF}(c,q) = \frac{1}{k + r_{\mathrm{bm25}}(c,q)} + \frac{1}{k + r_{\mathrm{dense}}(c,q)}, \qquad k = 60.$$

Only the ordinal ranks matter—no score calibration required. The offset $k = 60$ prevents rank-1 from contributing a disproportionately large 1/1 term; it acts as a floor that smooths the contribution of highly-ranked items.

Why $k = 60$? Cormack et al. (2009) found empirically that $k = 60$ is robust across many collection types; larger k gives equal weight to all results, smaller k amplifies top-ranked items. 60 is a practical default, not a derived constant.

When to use RRF. Whenever BM25 and dense retrieval are both available (the common hybrid RAG architecture). RRF is parameter-free beyond k, requiring no score normalisation—scores from BM25 (typically 0–30) and dense retrievers (cosine similarity, 0–1) are on incompatible scales but ranks are not.

- **Sparse vs dense retrieval:** lexical overlap and embedding similarity capture different relevance modes; hybrid retrieval via RRF combines both without score calibration.
- **Fusion methods:** rank fusion stabilizes retrieval when signals disagree.
- **Evaluation decomposition:** retrieval-stage errors and generation-stage errors require separate diagnostics.
- **Grounding metrics:** answer support must be checked against retrieved evidence, not linguistic plausibility.

How These Results Build on Each Other

1. Define relevance and evidence assumptions.

2. BM25 scores each chunk by IDF-weighted, length-normalised term frequency.
3. Dense retrieval scores each chunk by embedding cosine/dot product.
4. RRF fuses both ranked lists via reciprocal rank sum, no calibration required.
5. Measure retrieval coverage/precision (Precision@k, Recall@k, MRR).
6. Measure generation quality conditioned on retrieved context.
7. Measure grounding/faithfulness of final claims.
8. Localize failure stage before proposing fix.

Reminder Glossary (Term by Term)

- **Gold relevance set:** trusted set of truly relevant items.
- **Precision@k:** fraction of top-k retrieved items that are relevant.
- **Recall@k:** fraction of relevant items captured in top-k.
- **MRR (Mean Reciprocal Rank):** average of 1/rank of first relevant doc across queries; rewards surfacing one correct result quickly.
- **BM25:** Best Match 25; probabilistic term-weighting retrieval model with IDF, TF saturation, and length normalisation. The default sparse baseline.
- **IDF (Inverse Document Frequency):** log-scaled measure of term rarity; rare terms receive higher weight.
- **TF saturation:** BM25's sub-linear treatment of term frequency; a term repeated 10 times scores less than 10× a single occurrence.
- **Length normalisation (b):** BM25 parameter controlling penalty for longer documents; $b = 0.75$ is the standard default.
- **Dense retrieval:** embedding-based retrieval scoring chunks by dot product or cosine similarity to a query vector.
- **RRF (Reciprocal Rank Fusion):** rank-combination formula $1/(k + r_1) + 1/(k + r_2)$ that merges two ranked lists without score calibration; $k = 60$ default.
- **Hybrid retrieval:** architecture running both BM25 and dense retrieval in parallel, fusing results (typically via RRF).
- **Reranker:** second-stage scoring model for refined ranking.
- **Faithfulness:** proportion of answer claims supported by context.

Worked Mini Example (BM25 Single Term)

Corpus of $N = 1000$ documents; term "`fiduciary`" appears in df $= 5$ of them. Query $q = \{\texttt{fiduciary}\}$, chunk c has $\mathrm{TF}(t, c) = 2$, $|c| = 80$ tokens, avgdl $= 100$. Parameters: $k_1 = 1.5$, $b = 0.75$.

$$\mathrm{IDF} = \log\left(\frac{1000 - 5 + 0.5}{5 + 0.5} + 1\right) = \log(182.4) \approx 5.21.$$

$$\text{length-norm denom} = 1 - 0.75 + 0.75 \cdot \tfrac{80}{100} = 0.25 + 0.60 = 0.85.$$

$$\text{TF factor} = \frac{2 \cdot (1.5 + 1)}{2 + 1.5 \cdot 0.85} = \frac{5.0}{3.275} \approx 1.53.$$

$$\mathrm{BM25}(c, q) = 5.21 \times 1.53 \approx 7.97.$$

Key observations: the rare term (`fiduciary` in only 5/1000 docs) earns high IDF; the TF factor is saturated at 1.53 rather than the raw 2; the shorter-than-average chunk ($|c|/\text{avgdl} = 0.8$) receives a slight length bonus.

Worked Mini Example (RRF Fusion)

Three chunks retrieved by BM25 and a dense encoder. Ranks (1-indexed):

Chunk	r_{bm25}	r_{dense}
A	1	3
B	2	1
C	3	2

RRF scores with $k = 60$:

$$\mathrm{RRF}(A) = \tfrac{1}{61} + \tfrac{1}{63} \approx 0.0164 + 0.0159 = 0.0323,$$

$$\mathrm{RRF}(B) = \tfrac{1}{62} + \tfrac{1}{61} \approx 0.0161 + 0.0164 = 0.0325,$$

$$\mathrm{RRF}(C) = \tfrac{1}{63} + \tfrac{1}{62} \approx 0.0159 + 0.0161 = 0.0320.$$

Final ranking: $B > A > C$. Chunk B wins by being rank-1 in dense retrieval—even though it was rank-2 in BM25—because the fusion treats both signals symmetrically. The margin is small: rank differences of 1–2 near the top of the list yield very similar RRF scores due to the $k = 60$ offset.

Worked Mini Example (Precision and Recall@k)

Suppose the gold relevant set has 4 items. Top-5 retrieval returns 3 relevant items.

$$\text{Precision@5} = \frac{3}{5} = 0.6, \qquad \text{Recall@5} = \frac{3}{4} = 0.75.$$

Interpretation: ranking quality and evidence coverage are both incomplete, but in different ways.

Worked Example (Failure Localization)

Pipeline A improves final answer score from 68 to 72. But decomposition reveals:

- retrieval recall@5 dropped from 0.81 to 0.70,
- generation conditioned on gold context improved substantially.

Interpretation: generator improved, retriever regressed. Action: fix retrieval stage rather than over-tuning generation prompts.

Intuition Checkpoints

- Better answer fluency can hide retrieval failures.
- High retrieval recall can coexist with poor final answers if synthesis is weak.
- Metric definitions require explicit edge-case conventions.
- BM25 is not a neural model — it has no learned parameters, runs in microseconds, and degrades gracefully on unseen terminology (the term just gets zero TF). That makes it robust for cold-start private corpora before any fine-tuning is possible.
- The TF saturation in BM25 is a deliberate design choice grounded in probability theory (Robertson–Spärck Jones weighting). It prevents one heavily repeated term from swamping the entire score.
- RRF's $k = 60$ offset means that rank 1 contributes 1/61 rather than 1/1: a roughly 60× smaller maximum score. This prevents any single system from dominating when results disagreeing strongly near the top.
- Hybrid retrieval (BM25 + dense + RRF) is the practical default for RAG systems because neither modality alone is sufficient: BM25 misses paraphrases; dense retrieval misses exact-match keywords and OOV terms.

Further Refresh

- Embedding geometry: see Chapter 4 refresher.
- Calibration/uncertainty interpretation: see Chapter 1 refresher.
- Systems throughput constraints: see Chapter 7 refresher.
- **Manning et al., *IIR*** — BM25, ranking metrics, IR evaluation.
- **FAISS/HNSW papers** — ANN retrieval behavior and tradeoffs.

Chapter 11

Chapter 11 Refresher: Governance, Assurance, and the Future of AI Systems — Risk Arithmetic and Assurance Logic

SETTING THE SCENE

Risk arithmetic translates qualitative hazards into inequalities you can monitor—probability returns as operational thresholds on empirical frequencies (Preface Steps 1–4 recycled for compliance narratives).

Step A — Hazards as events. Problem. Stakeholders disagree verbally unless Ω carved crisply. **Insight.** Define hazardous outcomes as measurable events; attach severities/likelihoods only after measurability clears. **Lineage.** Leveson STAMP lineage; ISO/IEC flavour standards.

Step B — Controls multiply imperfectly. Problem. Layered mitigations stack but rarely independent. **Insight.** Product-form residual models mirror naive independence yet expose correlation caveats—same scepticism as Preface Step 6 covariance warnings.

Step C — Evidence graphs. Problem. Auditors demand provenance. **Insight.** Assurance cases are Bayesian-flavoured narratives without always updating posteriors—still DAGs connecting claims to sensors.

Governance makes Preface probability language accountable to humans outside ML conferences.

Notation Introduced in This Chapter

- R: baseline risk score.
- R_{res}: residual risk after controls.
- η_i: efficacy of control i.
- m_t: monitored metric at time t.
- m_0: baseline metric value.
- ϵ: escalation threshold.

What You Need To Recall Precisely

- **Risk decomposition:** severity and likelihood are different axes with different uncertainty.
- **Layered controls.** With baseline scalar risk score R and independent-ish controls each attenuating fraction $\eta_i \in [0, 1]$,
$$R_{\text{res}} = R \prod_i (1 - \eta_i).$$
Interpret η_i as fractional reductions attributable to distinct safeguards under optimistic independence assumptions; shared vulnerabilities violate independence and make multiplicative models overly cheerful—inflate residuals mentally whenever controls depend on the same latent failures.
- **Trigger logic:** indicator functions implement threshold decisions.
- **Assurance structure:** claim → evidence → control action → artifact.
- **Correlation caveat:** multiplicative risk-reduction models are optimistic under shared-cause failures.

How These Results Build on Each Other

1. Define risk factors and their measurement status.
2. Model baseline and post-control residual risk.
3. Apply threshold policy to operational metrics.
4. Attach each decision to auditable evidence.

Reminder Glossary (Term by Term)

- **Hazard:** undesirable outcome class to prevent/mitigate.
- **Control efficacy:** estimated fractional risk reduction from a control.
- **Residual risk:** remaining risk after controls.
- **Threshold policy:** decision rule on monitored metrics.
- **Assurance artifact:** evidence item supporting a claim.

Worked Mini Example (Single vs Two Controls)

If baseline risk score is $R = 1.0$, and control efficacies are $\eta_1 = 0.4$, $\eta_2 = 0.3$:

$$R_1 = R(1 - \eta_1) = 0.6, \qquad R_{12} = R(1 - \eta_1)(1 - \eta_2) = 0.42.$$

Interpretation: layered controls reduce residual risk multiplicatively under the model assumptions.

Worked Example (Threshold and Escalation)

Suppose drift metric baseline is $m_0 = 0.12$ and policy threshold is $\epsilon = 0.05$. Current observed $m_t = 0.19$ gives deviation $|m_t - m_0| = 0.07 > \epsilon$. Escalation rule triggers high-review mode. Reasoning chain: measured metric -> threshold test -> policy action -> logged evidence.

Intuition Checkpoints

- Multiplicative control models can be optimistic if failures are correlated.
- Governance thresholds are policy design decisions, not immutable laws.
- Audit readiness is about traceable evidence chains, not only numeric scores.

Further Refresh

- Metric reliability and uncertainty: see Chapter 1 refresher.
- Evaluation decomposition and harness logic: see Chapter 10 refresher.
- **Leveson, *Engineering a Safer World*** — hazard/control argument structure.
- **NIST AI RMF** — operational governance controls and documentation patterns.

Chapter 12

Chapter 12 Refresher: Modern LLM Access — APIs, SDKs, Agents, and the Model Context Protocol — Latency, Queues, and Reliability Budgets

SETTING THE SCENE

APIs expose stochastic models inside stochastic infrastructure: arrivals, service times, retries, and pricing multiply Preface Steps 5–7 expectations into tail experiences users actually feel.

Step A — Utilisation as latent probability. Problem. Saturated GPUs inflate latency although average load looks fine. **Insight.** $\rho = \lambda/\mu$ nearing 1 blows variance (Preface Step 6)—queue waits explode superlinearly. **Lineage.** Kendall queue notation; Kleinrock textbooks. **Mental model.** Treat ρ like criticality in statistical mechanics—small deltas matter near phase transitions.

Step B — Retries compound load. Problem. Clients retry on flakes. **Insight.** Geometric retry counts lift effective λ—same algebra as Preface Step 4 inverted conditioning stories but operational.

Step C — Guards cap rare disasters. Problem. Agents might loop unbounded. **Insight.** Hard budgets reinstate finite Ω partitions compatible with Step 1 measurability mindset.

Probability-minded reliability engineers read LM dashboards the same way traders read risk tapes—moments plus tails.

Notation Introduced in This Chapter

- λ: request arrival rate.
- μ: service rate.
- $\rho = \lambda/\mu$: utilization.
- $\mathbb{E}[W]$: expected waiting time.

- p_f: per-attempt failure probability.
- $\mathbb{E}[K]$: expected number of attempts.

What You Need To Recall Precisely

- **Latency decomposition:** prefill latency plus per-output-token decode latency.
- **Queueing baseline (M/M/1).** Assume Poisson arrivals rate λ, exponential service rate μ per busy server, single server, infinite buffer (steady-state regime $\lambda < \mu$). Expected waiting time in queue satisfies
$$\mathbb{E}[W] = \frac{1}{\mu - \lambda},$$
blowing up as $\rho = \lambda/\mu \uparrow 1$: tiny overload deltas dominate latency tails—engineering intuition beats pretending arrivals are perfectly deterministic.
- **Retry expectation.** If each attempt fails independently with probability p_f and retries repeat until first success (no cap), successes follow geometric distribution with success probability $1 - p_f$. Expected attempts $\mathbb{E}[K] = \sum_{n \geq 1} n\, p_f^{n-1}(1 - p_f) = 1/(1 - p_f)$. Even modest p_f inflates load multipliers $\mathbb{E}[K]$ nonlinearly.
- **Token cost model:** base cost is linear in tokens, expected cost depends on retries and aborts.
- **Termination guards:** bounded steps/tokens/cost enforce finite agent behavior.

How These Results Build on Each Other

1. Estimate traffic intensity (λ/μ).
2. Predict queue delay and saturation risk.
3. Incorporate retries into effective load and expected cost.
4. Enforce runtime guards for bounded execution.

Reminder Glossary (Term by Term)

- **Arrival rate λ:** requests per unit time.
- **Service rate μ:** processing capacity per unit time.
- **Utilization ρ:** λ/μ.
- **Retry multiplier:** expected attempts per successful request.
- **Budget guard:** hard bound on cost/tokens/steps.

Worked Mini Example (Retries)

Let per-attempt failure probability be $p_f = 0.2$ and assume independent retries with no cap.

$$\mathbb{E}[K] = \frac{1}{1 - p_f} = \frac{1}{0.8} = 1.25.$$

If average successful request cost is \$0.04, expected cost with retries is roughly

$$0.04 \times 1.25 = \$0.05.$$

Worked Example (Queue and Retry Coupling)

If base arrival rate is 80 req/s and retries add 25% expected attempts, effective arrival becomes:

$$\lambda_{\text{eff}} = 80 \times 1.25 = 100 \text{ req/s}.$$

If service rate is $\mu = 110$ req/s, utilization rises from 0.73 to 0.91. Interpretation: retry policy can push system into tail-latency regime even when baseline looked safe.

Intuition Checkpoints

- Systems can meet average SLOs while failing badly in tails.
- Queueing approximations are useful planning tools, not exact traffic simulators.
- Reliability controls (backoff, circuit breaking) alter both latency and cost.

Further Refresh

- Systems scaling and bottlenecks: see Chapter 7 refresher.
- Evaluation and trust boundaries: see Chapter 10 refresher.
- Risk thresholding and controls: see Chapter 11 refresher.
- **Harchol-Balter** — queueing and latency scaling.
- **SRE workbooks** — retries/backoff/overload control design.

Chapter 13

Chapter 13 Refresher: A Human-Centric Model for Understanding LLM Errors — Diagnostic Decomposition and Multi-Objective Reasoning

SETTING THE SCENE

Closing the loop returns evaluation to Preface discipline: partition failures into measurable slices before averaging hides pathology (Step 7 scepticism about naive empirical means).

Step A — Axes as orthogonal-ish hypotheses. Problem. "Bad answer" is ambiguous utterance. **Insight.** Five axes mimic independent latent causes though reality correlates them—approximate factorial design for debugging. **Lineage.** Human-factors evaluation tradition; reader-specific ladder framing.

Step B — Diagnostics precede optimisation. Problem. Teams tweak prompts when retrieval broke. **Insight.** Hypothesis testing instinct from Preface Step 4 / classical experimentation—rule out cheap structural failures before blaming semantics.

Step C — Multi-objective feasibility. Problem. Accuracy, safety, latency, cost trade off. **Insight.** Pareto surfaces generalise scalar loss discomfort from Chapter 2—no magic scalar unless governance chooses weights.

The axes listed operationally:

- **Structure:** grammar/schema/tool-protocol adherence.
- **Meaning:** semantic alignment despite fluency.
- **Context:** prompt/task cue fidelity.
- **Grounding:** evidence-backed claims.
- **Environment:** tool/SDK behavioural assumptions.

Structured diagnosis honours Preface probability hygiene: define events precisely, measure their frequencies, intervene where measurements spike.

Notation Introduced in This Chapter

- Axis labels: structure, meaning, context, grounding, environment.
- Conditional slice: subset of cases defined by context/task conditions.
- Tradeoff surface: set of feasible points across accuracy, safety, latency, and cost.

What You Need To Recall Precisely

- **Failure-axis decomposition:** structure, meaning, context, grounding, environment are separate tests.
- **Conditional error analysis:** inspect slices and contexts, not only aggregate rates.
- **Correlation vs intervention:** observed association does not imply remediation leverage.
- **Multi-objective tradeoffs:** optimize over accuracy, safety, latency, and cost jointly.
- **Diagnostic ordering:** simple structural failures should be ruled out before attributing deep semantic or governance causes.

How These Results Build on Each Other

1. Observe failure instance and classify candidate axes.
2. Check low-level structural and context conditions first.
3. Test grounding and environment causes next.
4. Select intervention matched to diagnosed failure pathway.

Reminder Glossary (Term by Term)

- **Structure failure:** format/syntax/protocol violation.
- **Meaning failure:** semantic mismatch despite fluent output.
- **Context failure:** missing or misused prompt/task constraints.
- **Grounding failure:** unsupported factual claims.
- **Environment failure:** external system/tool mismatch.

Worked Mini Example (Axis Decision)

Suppose an answer is fluent but cites a nonexistent document.

- **Structure:** pass (syntax and format are valid).
- **Meaning:** partial pass (language coherent).
- **Grounding:** fail (claim cannot be verified against source evidence).

The corrective action should prioritize grounding controls (retrieval/citation verification), not merely prompt rephrasing.

Worked Example (Axis-Matched Intervention)

Case: answer includes correct format and plausible language but wrong regulation citation.

- Structure: pass.
- Meaning: partial pass.
- Grounding: fail (citation not verifiable).

Intervention path:

1. add retrieval requirement from authoritative corpus,
2. enforce citation-span verification before final response,
3. reject answer if citation cannot be matched.

Reasoning: this is not primarily a prompt-style failure; it is evidence-path failure.

Intuition Checkpoints

- A plausible answer can still be ungrounded.
- One failure can belong to multiple axes; diagnosis is often multi-label.
- The strongest fix is the one that changes the relevant information pathway.

Pointers

- Calibration and uncertainty foundations: see Chapter 1 refresher.

Further Refresh

- Evaluation slicing and retrieval grounding: see Chapter 10 refresher.
- Risk and assurance framing: see Chapter 11 refresher.
- Runtime constraints and failure budgets: see Chapter 12 refresher.
- **Murphy** — probabilistic uncertainty reasoning.
- **Error analysis/evaluation notes** — slicing and diagnostic methodology.